

Outline:

- Introduction to Services Mesh
- Different services of modern Istio Service Mesh

Study environment:

- Lab materials are present on Moodle
- In our laboratory we will use Kubernetes cluster and tools from previous laboratory
- Install *istioctl* tool (<https://istio.io/latest/docs/setup/getting-started/>). Use the latest version.
- The laboratory is based on exercises prepared by Bartosz Żurkowski (<https://istioworkshop.github.io/>)
- Please combine the results of all tasks in one single pdf file and attach it in UPEL

Introduction

The workshop's **primary objective is to help you acquire practical experience in the administration and usage of Istio service mesh**. The course will guide you through the most common practical use-cases such as service monitoring, traffic routing, canary releases, and fault injection. In addition, the course will guide you through the installation of a Kubernetes cluster and Istio, which are the prerequisites of the workshop.

Each section provides a short introduction to familiarize yourself with a feature or a problem and then gives step-by-step instructions describing how to achieve the goal.

Some chapters contain tasks for individual practice. They will be listed at the end of the chapter under the **Exercises** section. You are obliged to prepare a concise report summarizing the completed tasks. Please, include executed commands, applied configuration policies, or screenshots depending on the task type. Use your best judgment to decide what is essential to include in the report - there is no need to include everything. The recommended report format is PDF.

The entire workshop will be conducted on a Kubernetes cluster launched using AWS Academy services. The number of credits allocated to your account should be sufficient for the cluster to operate for a longer time, but remember to be economical. Try to complete all instructions in one run. If this is not possible, please reset account after the each attempt.

Finally, the course assumes that you have basic knowledge about Service Mesh and Istio. If you are not there yet, first read the following two pages from the Istio documentation:

- [What is Istio?](#)
- [Istio traffic management](#)
- [Architecture](#)

Task -1 – Prepare and Verify your environment

First, using AWS Console create cluster in Elastic Kubernetes Service (in search dialog type EKS)

Networking [Info](#)

IP address family and service IP address range cannot be changed after cluster creation.

VPC [Info](#)

Select a VPC to use for your EKS cluster resources. To create a new VPC, go to the [VPC console](#).

vpc-02aff30987ef91d6f | Default

Subnets [Info](#)

Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets

subnet-0b0d7b54aa1221a71

X

us-east-1a 172.31.16.0/20

subnet-00bf4ab901facb5fe

X

us-east-1b 172.31.32.0/20

Clear selected subnets

Security groups [Info](#)

Choose the security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups

Choose cluster IP address family [Info](#)

Specify the IP address type for pods and services in your cluster.

☒ IPv4

☐ IPv6

☐ Configure Kubernetes service IP address range [Info](#)

Specify the range from which cluster services will receive IP addresses.

Once it is done, add nodes to cluster. In the cluster options add nodes to Node group.

[EKS](#) > [Clusters](#) > suu-cluster

suu-cluster

Delete cluster

▼ Cluster info [Info](#)

Status

Active

Kubernetes version [Info](#)

1.29

Support type

Standard support until March 23, 2025

Provider

EKS

Overview

Resources

Compute

Networking

Add-ons

Access

Observability

Upgrade insights

Update history

Tags

Nodes (0) [Info](#)

Filter Nodes by property or value

< 1 >

Node name

Instance type

Node group

Created

Status

No Nodes

This cluster does not have any Nodes, or you don't have permission to view them.

Node groups (0) [Info](#)

Edit

Delete

Add node group

Group name

Desired size

AMI release version

Launch template

Status

No node groups

This cluster does not have any node groups.

Nodes that are not part of an Amazon EKS managed node group are not shown in the AWS console.

Add node group

Configure node group [Info](#)

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name
Assign a unique name for this node group.

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role [Info](#)
Select the IAM role that will be used by the nodes. To create a new role, go to the [IAM console](#).

i The selected role must not be used by a self-managed node group as this could lead to a service interruption upon managed node group deletion.

[Learn more](#)

EKS > Clusters > suu-cluster

suu-cluster ⌂ Delete cluster

▼ **Cluster info** [Info](#)

Status Active	Kubernetes version Info 1.29	Support type Standard support until March 23, 2025	Provider EKS
------------------	---	---	-----------------

Overview Resources **Compute** Networking Add-ons Access Observability Upgrade insights Update history Tags

Nodes (2) [Info](#)

Filter Nodes by property or value

Node name	Instance type	Node group	Created	Status
ip-172-31-23-41.ec2.internal	t3.medium	suu-nodes	Created 2 minutes ago	Ready
ip-172-31-46-148.ec2.internal	t3.medium	suu-nodes	Created 2 minutes ago	Ready

Node groups (1) [Info](#) Edit Delete Add node group

Group name	Desired size	AMI release version	Launch template	Status
suu-nodes	2	1.29.0-20240409	-	Active

Next provision the cluster. First setup AWS account on your local machine, please perform following steps:

1. You can create environment by typing *aws configure*
 - a. Add at least one data to config
2. Copy your AWS credentials from Vocareum site to local machine. Store them in `~/.aws/credentials`.
3. Create default values for session:


```
aws configure set default.region us-east-1
aws configure set default.output table
```
4. Check the environment (there should be no ec2 instances and no errors on console):


```
aws sts get-caller-identity
```

Next configure the kubectl command

```
aws eks describe-cluster --region us-east-1 --name <your cluster> --query cluster.status
aws eks --region us-east-1 update-kubeconfig --name <your cluster>
```

You should see info that the cluster is ready. Now check the status of nodes:

kubectl get nodes

Also check that all Kubernetes components are ready (here we are using *kube-system* namespace):

kubectl --namespace kube-system get pods

TASK -1 – list all nodes and components in your cluster (the same as on previous labs)

Task 0 – Install Istio Control Plane

Istio provides an advanced tool named *istioctl* for managing its control and data planes. The tool enables automated installation of Istio with custom installation profiles. Besides, it provides a set of commands for inspecting the configuration of individual sidecars and troubleshooting issues that emerge in the service mesh.

We will use the *istioctl* binary multiple times throughout the workshop. Use the *istioctl help* command whenever you encounter issues. The command output will provide you with all necessary instructions (options with detailed documentation). For instance: *istioctl help manifest apply*.

In order to install Istio control plane, execute *istioctl manifest apply* command:

istioctl manifest apply --set profile=demo

Note: the install should point to version 1.21

```
marekko@k8s-aws: $ istioctl manifest apply --set profile=demo
This will install the Istio 1.21.2 "demo" profile (with components: Istio core, Istiod, Ingress gateways, and Egress gateways) into the cluster. Proceed? (y/N) y
✓ Istio core installed
✓ Istiod installed
✓ Ingress gateways installed
✓ Egress gateways installed
✓ Installation complete
Made this installation the default for injection and validation.
```

The command will deploy all Istio components required for the purpose of the workshop:

- Istiod - key component, a composite of:
 - Istio Pilot - service discovery and config distribution to Envoy sidecars,
 - Istio Galley - validation of user-provided policies,
 - Istio Citadel - service mesh CA, certificate signing and distribution,
 - Istio Sidecar Injector - automated injection of sidecars into pods;
- Istio Ingress Gateway - ingress traffic control,
- Istio Egress Gateway - egress traffic control,
- Istio Tracing - distributed tracing, wrapper on top of Jaeger
- Prometheus - metric sink, collects and stores service metrics,
- Grafana - metric data visualization,
- Kiali - mesh topology dashboard.

Ensure that all deployed pods are running (inspect READY column):

kubectl -n istio-system get pods

kubectl -n istio-system get svc

Ensure that an External IP has been assigned to the Istio Ingress Gateway (EXTERNAL-IP column):

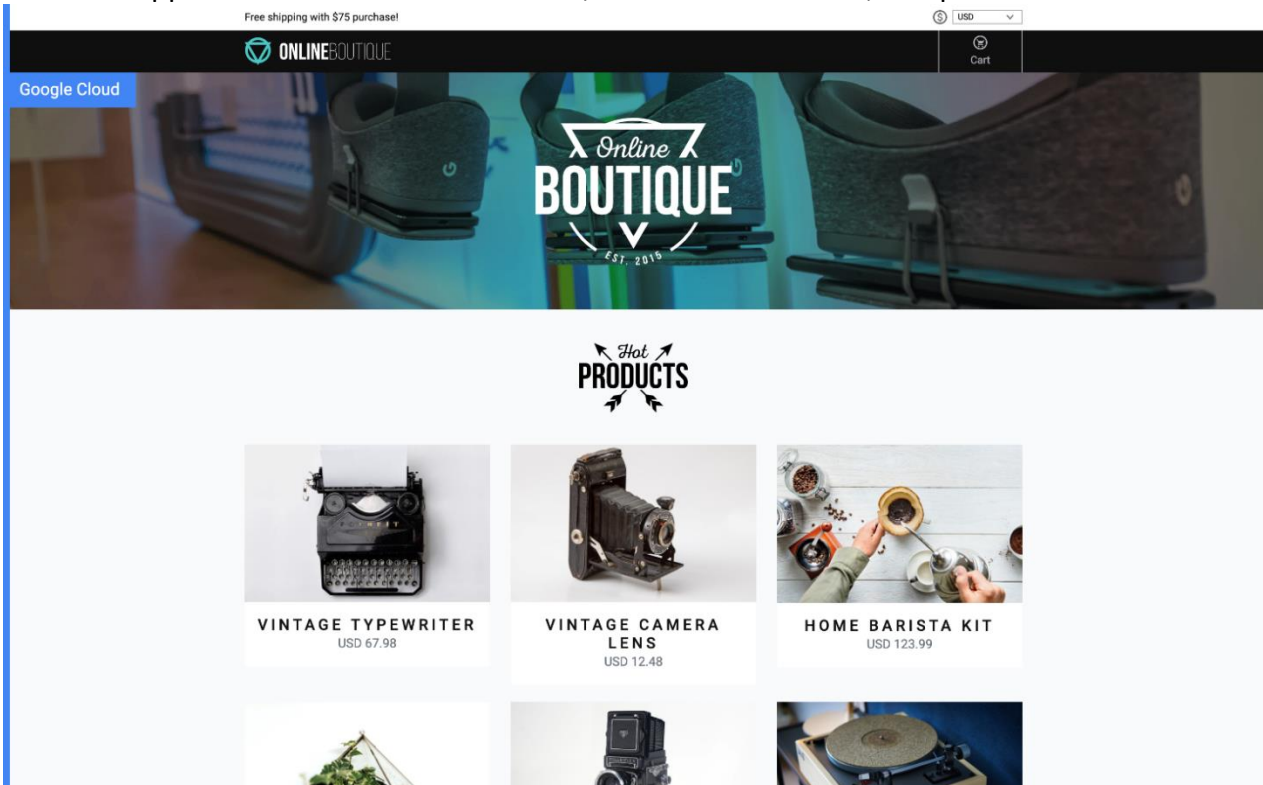
kubectl -n istio-system get svc istio-ingressgateway

TASK 0 – copy output of the last three commands to the report

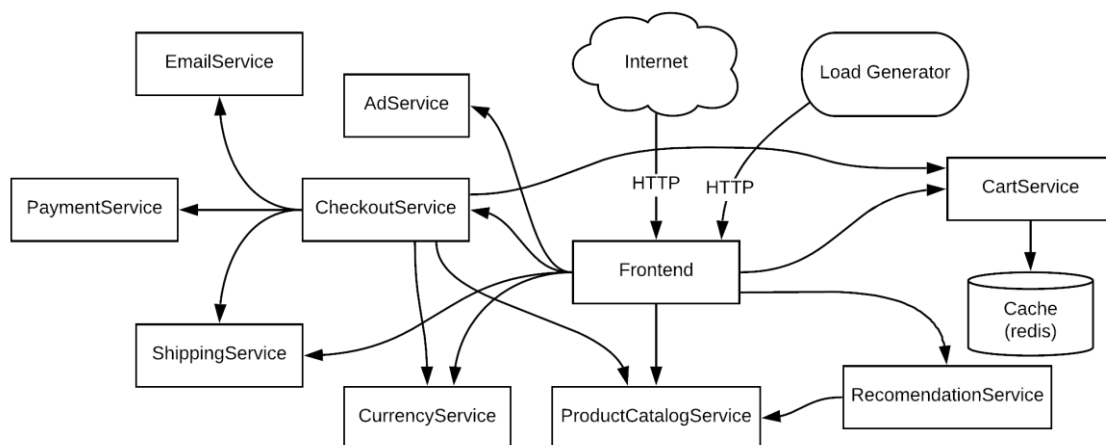
Task 1 – Deploy test application

For the purpose of this workshop we will use the test application named Online Boutique (previously: Hipster Shop) provided by the Google Cloud Platform team. The application will allow us to evaluate several service mesh scenarios, including traffic routing, and traffic shifting, and circuit breaking.

Online Boutique is a cloud-native microservices demo. It consists of 10 microservices, written in polyglot programming languages (Go, C#, Node.js, Python, Java), that implement a web-based e-commerce app where users can browse items, add them to the cart, and purchase them:



The application architecture is presented below:



The frontend service is an entry point to the application. It exposes an HTTP endpoint for its clients (web browser, load generator), providing the content compiled from the integrated microservices. Internally, the microservices communicate using the gRPC protocol.

The load generator populates random requests to the application to simulate user behavior. That enables evaluating service mesh scenarios under load.

In order to install the app lets start with Start with enabling the sidecar injection for the default namespace:

```
kubectl label namespace default istio-injection=enabled
```

The *istio-injection* label informs the Istio Sidecar Injector running in Istio control plane to inject a sidecar into each new pod created in the default namespace.

Sidecars running alongside application services are responsible for processing the traffic entering and leaving the pods. They act as local proxies capable of enforcing the requested traffic policies. Sidecars are the key building block required to form a service mesh. We will inspect them later.

Now, deploy the test application (use the files stored in the *template.zip* on UPEL – in directory templates):

```
kubectl apply -f templates/release/kubernetes/manifests.yaml
```

Note that some application services are deployed in multiple versions, for instance, emailservice and shippingservice.

Ensure that all deployed pods are running (inspect READY column):

```
kubectl -n default get pods
```

TASK 1 List all components in the default namespace (output of the last command)

Task 2 – Observability, service metrics

Most of the service mesh metrics are collected through the presence of a sidecar proxy installed in each application pod.

Each proxy generates a rich set of metrics describing the inbound and outbound traffic passing through the proxy. They also provide detailed statistics about the administrative functions of the proxy itself, including configuration and health information.

In addition, Istio provides a set of aggregated service-oriented metrics for monitoring inter-service communication. These metrics cover the four fundamental service health indicators: latency, traffic, errors, and saturation.

The Istio control plane also provides a collection of self-monitoring metrics. These metrics allow monitoring the behavior of Istio itself. In order to use them you need to install Graphana and Prometheus addons to Istio:

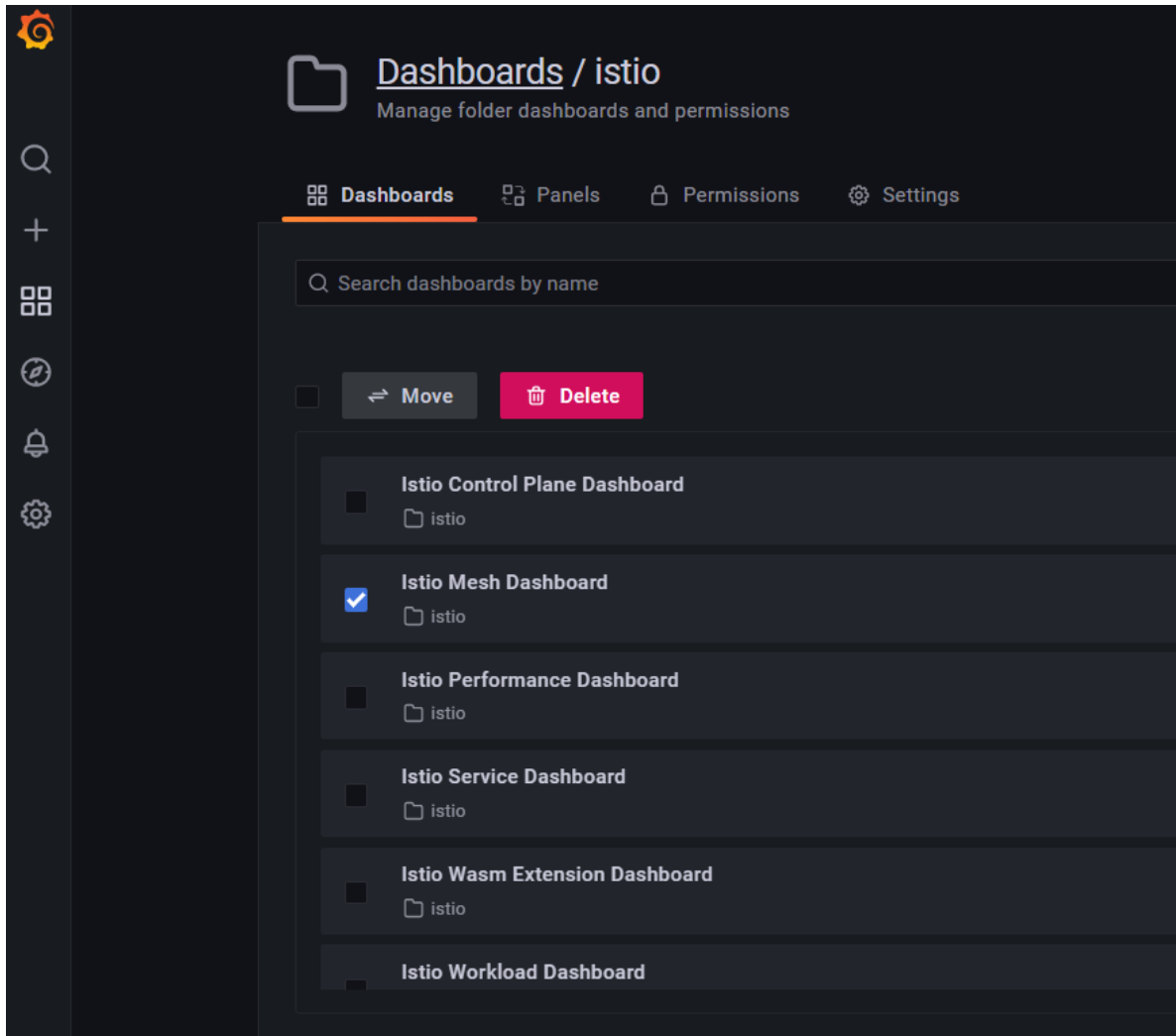
```
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.21/samples/addons/prometheus.yaml
```

```
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.21/samples/addons/grafana.yaml
```

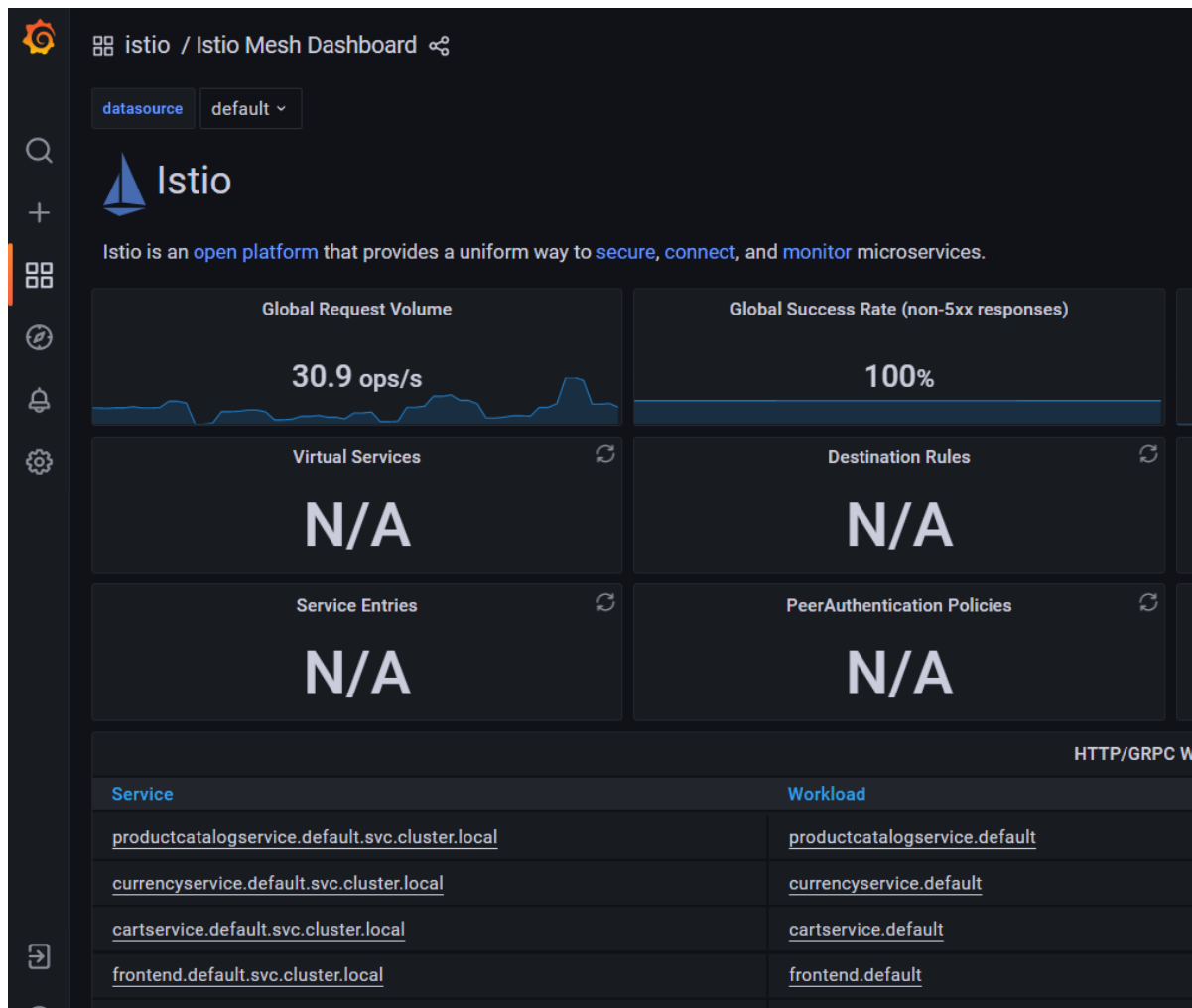
Port-forward the Grafana dashboard to your local machine:

```
istioctl dashboard grafana
```

You should be able to see add Istio metrics:



And finally see following screen:



The dashboard provides the global view of the service mesh:

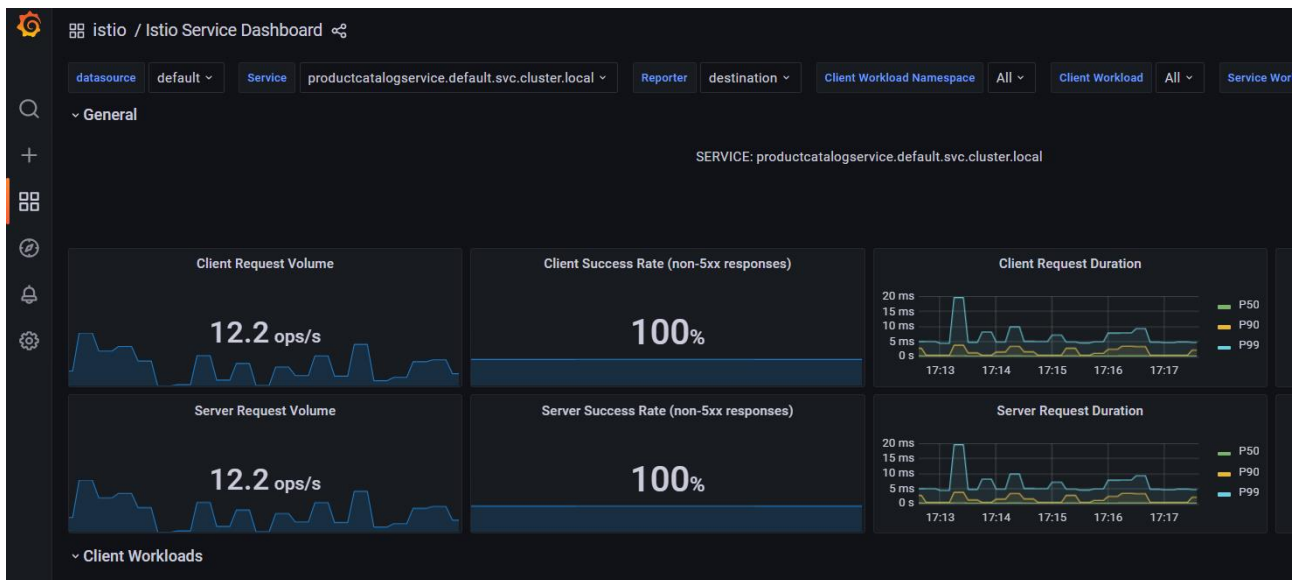
- total number of discovered services,
- global request volume expressed in operations per second (ops),
- global success rate - percentage of requests completed with success,
- total failed requests - number of requests ended with 4xx and 5xx HTTP error codes.

In addition, the dashboard provides essential metrics for the discovered services:

- operations per second,
- p50, p90, p99 latency,
- success rate.

Next navigate to Istio Service Dashboard pick the *productcatalog* service from the *Service* dropdown.

The dashboard should look similar to:

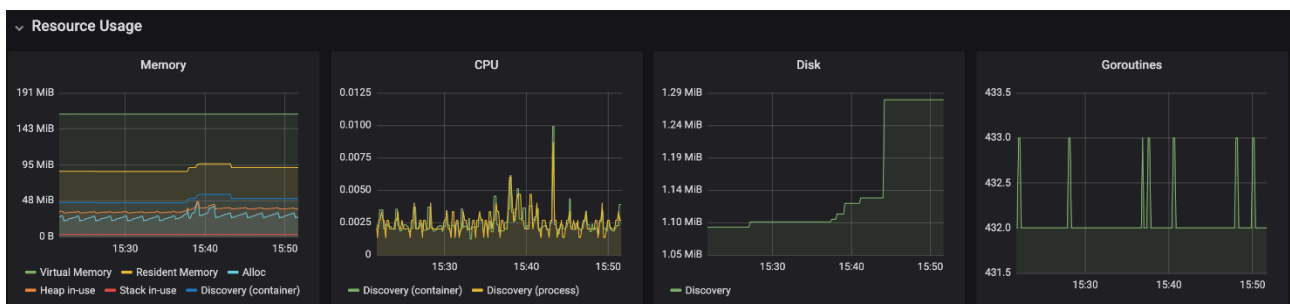


The dashboard provides detailed metrics for the selected service:

- client/server request volume,
- client/server success rate,
- client/server request duration,
- incoming requests by source,
- incoming success rate by source,
- incoming request duration by source,
- incoming request size by source,
- TCP received/sent bytes (only for raw TCP traffic).

Note that you can use the Service dropdown to switch between inspected services. Moreover, you can view the metrics in the context of a particular service client by using the Client workload dropdown.

Navigate to the **Istio Control Plane Dashboard**. It provides metrics related to Istio Pilot component deployed in the Istio control plane. The first part of the dashboard describes the usage of system resources (CPU, memory, disk):



The second part presents metrics specific to Pilot operation, for instance:

- Pilot pushes - frequency of configuration distribution to sidecar proxies,
- Pilot errors - number of errors encountered during configuration rendering and distribution,
- Proxy push time - time needed to synchronize configuration to a proxy.



Note, each component in the control plane provides a similar monitoring dashboard:

TASK 2 – Use the presented Grafana dashboards to answer the following questions (add them to report):

1. How many ops are being currently processed by the application?
2. What is the global success rate?
3. Are there any request errors occurring in the application at the moment? How do you check that?
4. Which application service processes the most requests? What is its p99 latency and success rate?
5. What is the current CPU and memory usage by Istio Pilot?
6. Does Istio Pilot encounter any errors at the moment?

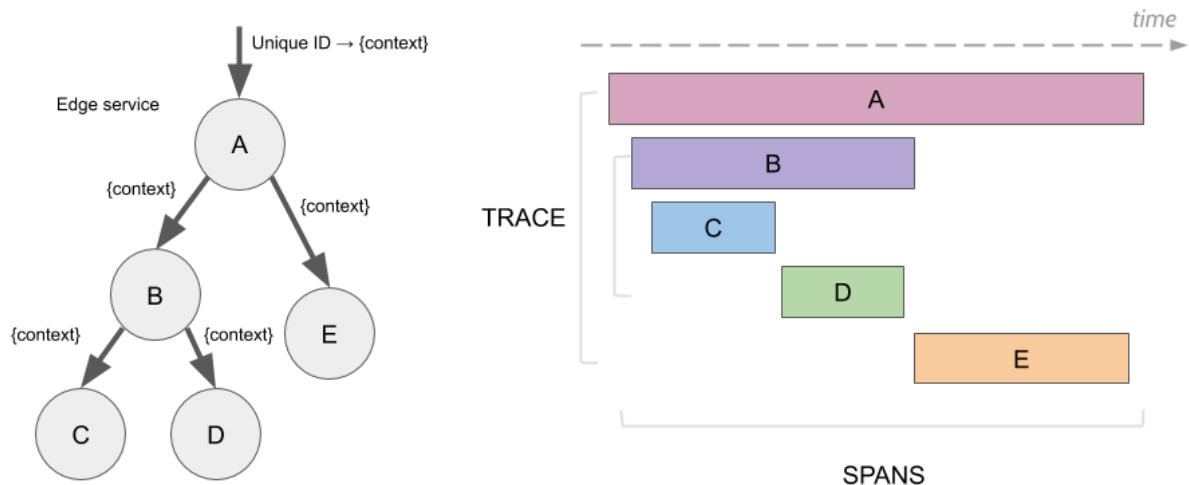
Task 3 – Observability, distributed tracing

Distributed tracing is a method used to profile and monitor distributed applications, especially those built using the microservices architecture.

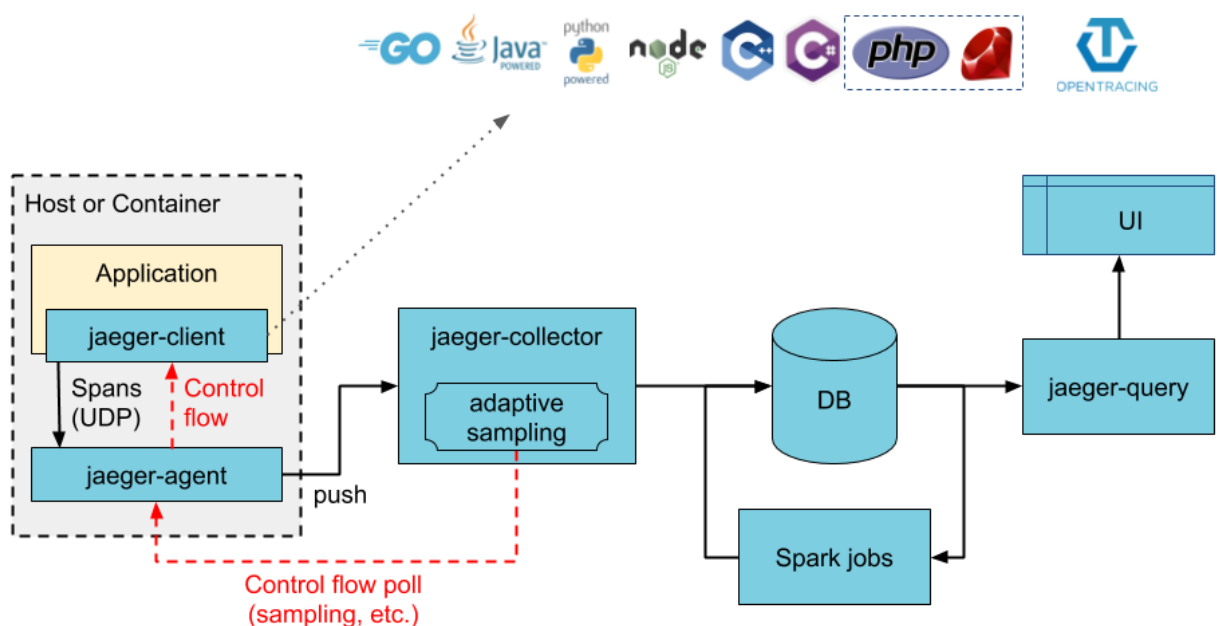
Requests in microservice-based applications often span multiple services. Each service handles a request by performing one or more nested operations, for instance, database queries or call to another service. In case something goes wrong, it is challenging to locate the failure in a large network of services. Similarly, it is difficult to identify a service that introduces a bottleneck (e.g., latency) in a long chain of interacting services.

Tools such as Jaeger or Zipkin compliant with OpenTracing standard help operators to analyze the request execution throughout the application.

In a nutshell, each initiated request is assigned with a unique request ID. The ID is propagated in request headers between services handling the request. Each service on the request path extracts the ID from the header, creates its request span by adding related context information, and sends the span to the local agent. The agent, in turn, groups spans by the request ID and forwards the aggregated data to the collector. The collector performs the final construction of the request trace.



In order to instrument the span generation and delivery to the agent, the application must use a dedicated tracing library. The library poses a need for implementation in a specific programming language, as well as requires configuration and development effort.



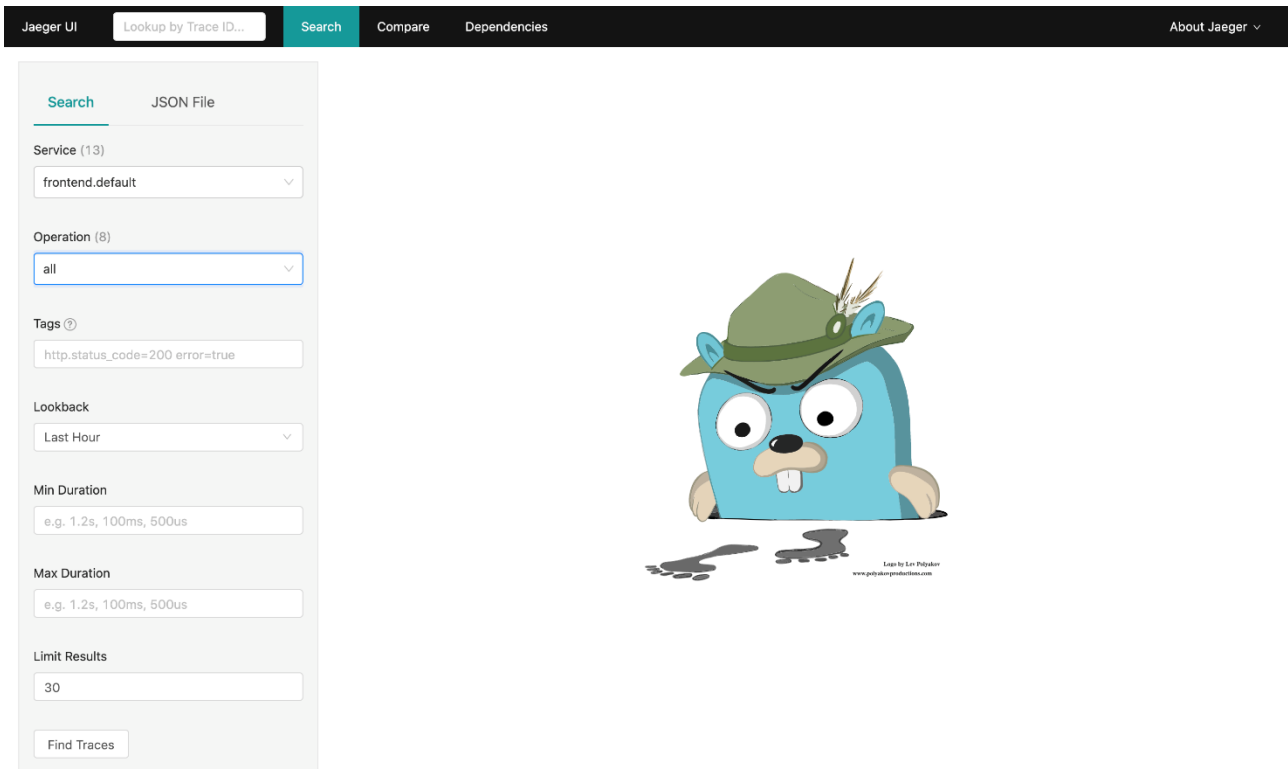
Next install Jaeger on your cluster:

```
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.21/samples/addons/jaeger.yaml
```

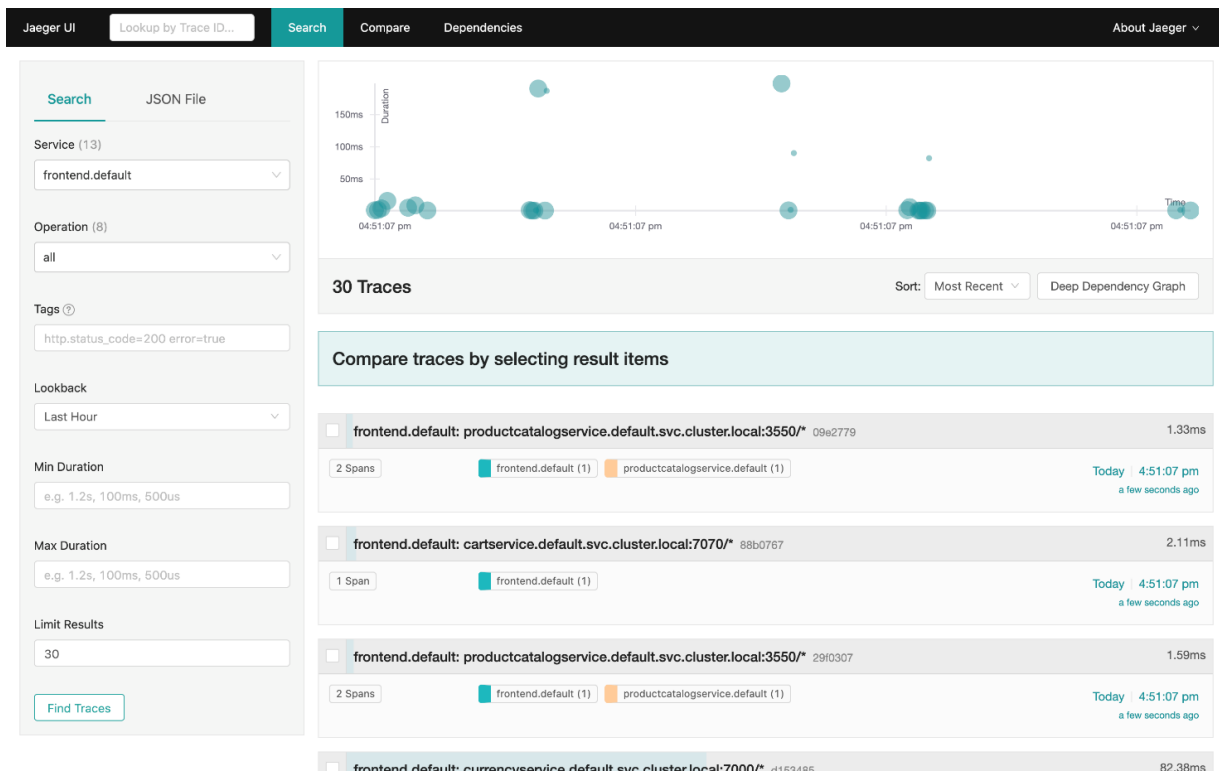
Port-forward the Jaeger dashboard to your local machine:

```
istioctl dashboard jaeger
```

Your browser should startup automatically and display the dashboard. Otherwise, visit the address: <http://localhost:16686>.



Search traces from the last hour for the frontend service. Limit the results to 30 traces. The results should be similar to:

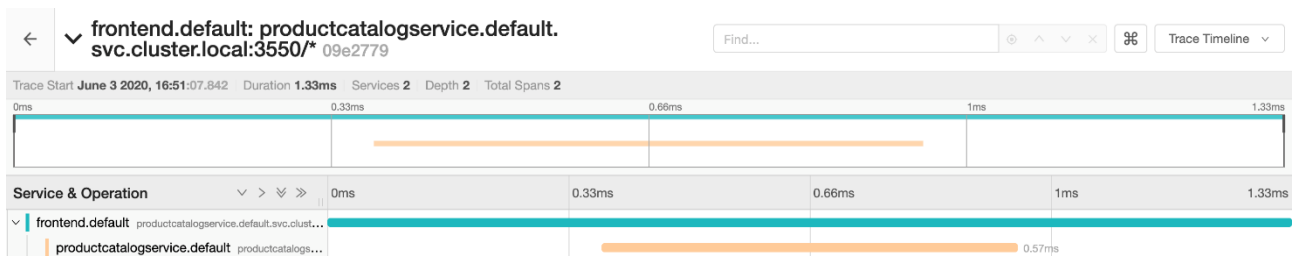


The plot at the top presents the clustered histogram of request traces over time. Below, Jaeger presents a list of captured request traces.

Trace represents a complete request path through the application (i.e., multiple microservices). It is composed of request spans reported by individual application services. Each span informs how much time a given service spent on processing the request and what was the request result (success or error).

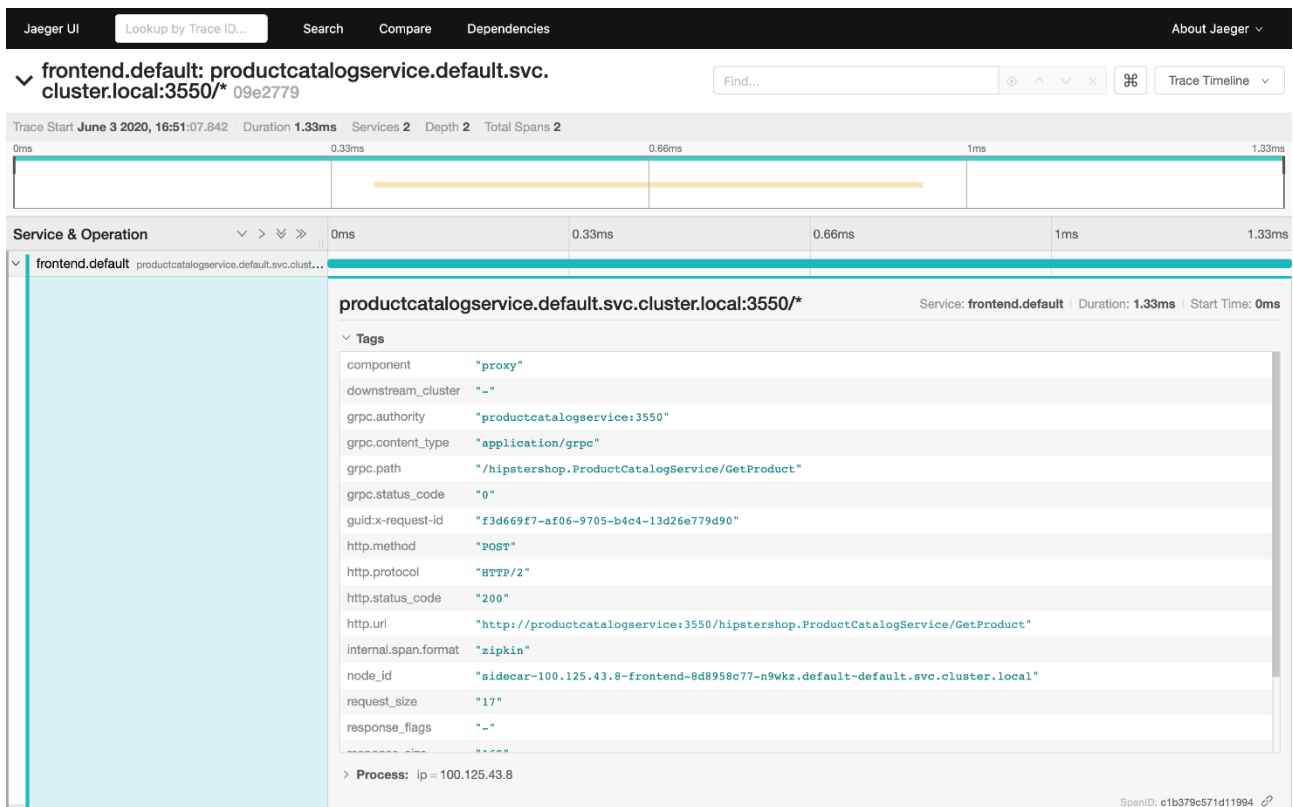
That way we can easily infer which service introduces a bottleneck (e.g., high latency) or which service causes a failure in a large application.

Let's inspect an arbitrary trace of length >1:



The above example illustrates trace of a request handled by two services: the frontend service calls the *productcatalog* service. The entire request took 1.33ms to complete, but 0.57ms of the request was handled by the *productcatalog* service.

Now, click on the frontend span to reveal the context information:



Istio proxy running in each application pod enhances request spans with rich context information:

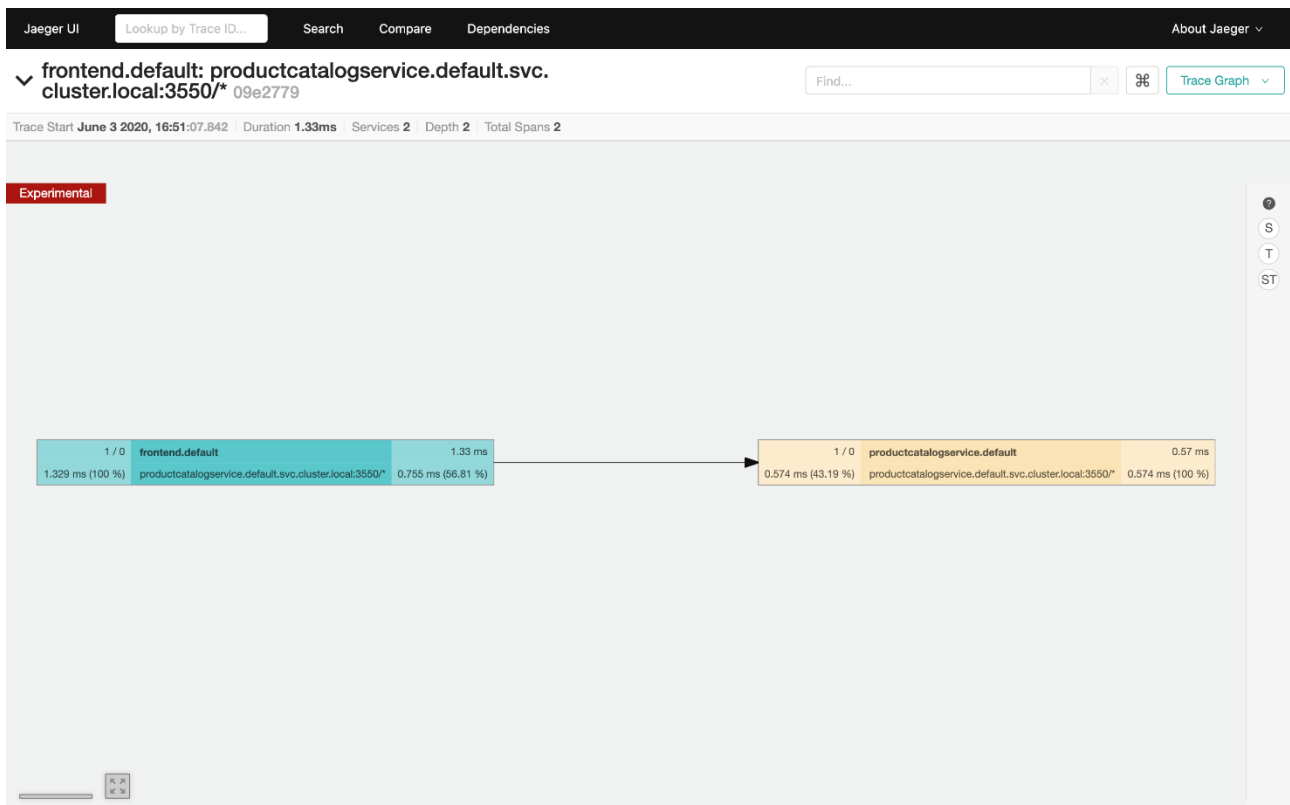
- content type,
- HTTP method,
- HTTP protocol,
- HTTP status code,
- request URL,
- request size,
- user agent.

These data can be useful for debugging the particular microservice identified as anomalous.

Note that without a proxy, the application would have to build the span (retrieve trace ID from request headers, inject context information) and sent it to the span collector on its own. That requires using a tracing library compatible with application programming language.

Since the proxy participates in all communication between services, it can offload the application from the tracing function. The tracing is transparent for the application and requires no configuration.

Lastly, switch to the graph representation of the trace:



The graph view enables you to inspect complex request traces consisting of many nested spans.

TASK 3 – Describe three distinct request paths from checkout service to other services. Include:

1. when the request started (date, time),
2. total request duration,
3. request protocol, method and status code,
4. duration of nested span.

Add everything to the report

Task 4 – Observability, mesh topology

Kiali is a management console for Istio service mesh with strong observability, configuration and validation capabilities. It provides a dashboard visualizing the mesh topology with rich information about inter-service dependencies and traffic health (latency, errors). Moreover, Kiali enables inspecting and producing service mesh configuration via a web interface and observing how the updated configuration affects performance of individual application services.

Kiali helps to answer the following questions:

- Which microservices are part of my service mesh?
- How are they connected?
- How are they performing?
- How can I operate on them?

Next install kiali on the cluster:

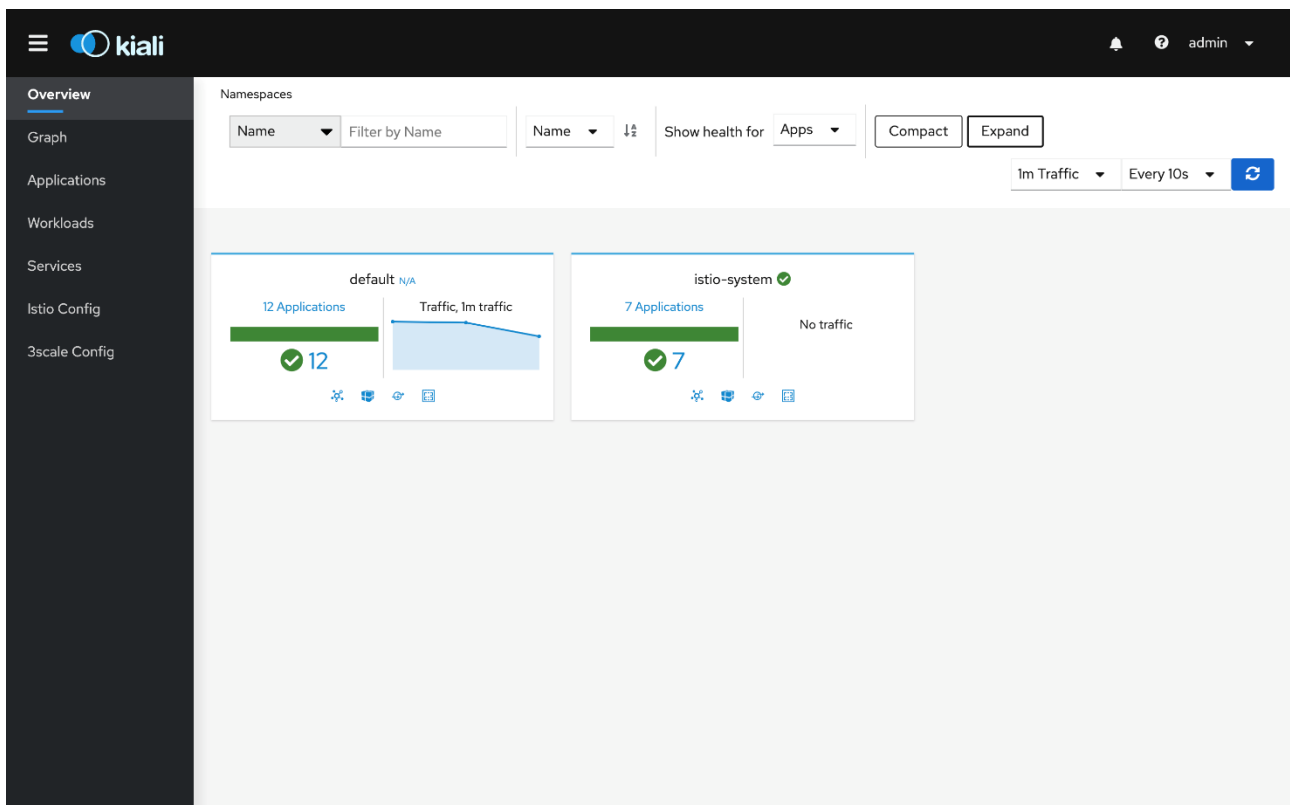
```
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.21/samples/addons/kiali.yaml
```

Port-forward the Kiali dashboard to your local machine:

```
istioctl dashboard kiali
```

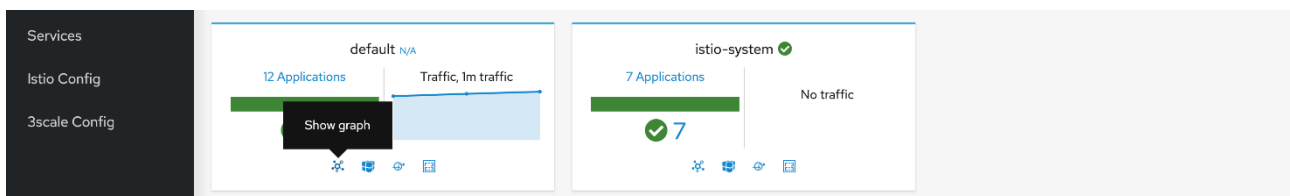
Your browser should startup automatically and display the dashboard. Otherwise, visit the address: <http://localhost:20001>.

After logging in, the dashboard should present the *Overview* page, similar to:

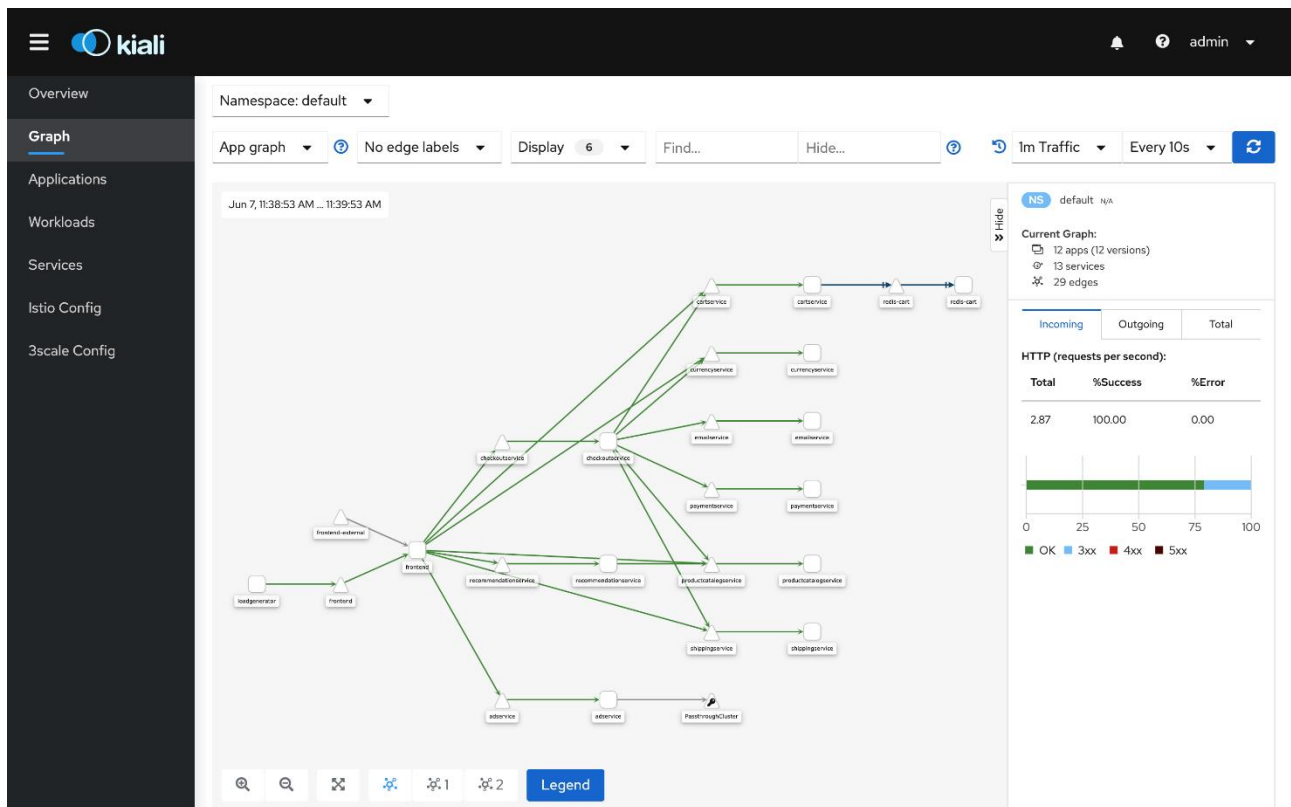


The page summarizes all Kubernetes namespaces that have services registered in the service mesh. In other words: namespaces that contain pods with Istio proxy.

Click on the graph icon in the default namespace card. The graph icon is in the lower left of the namespace card and looks like a connected group of circles:



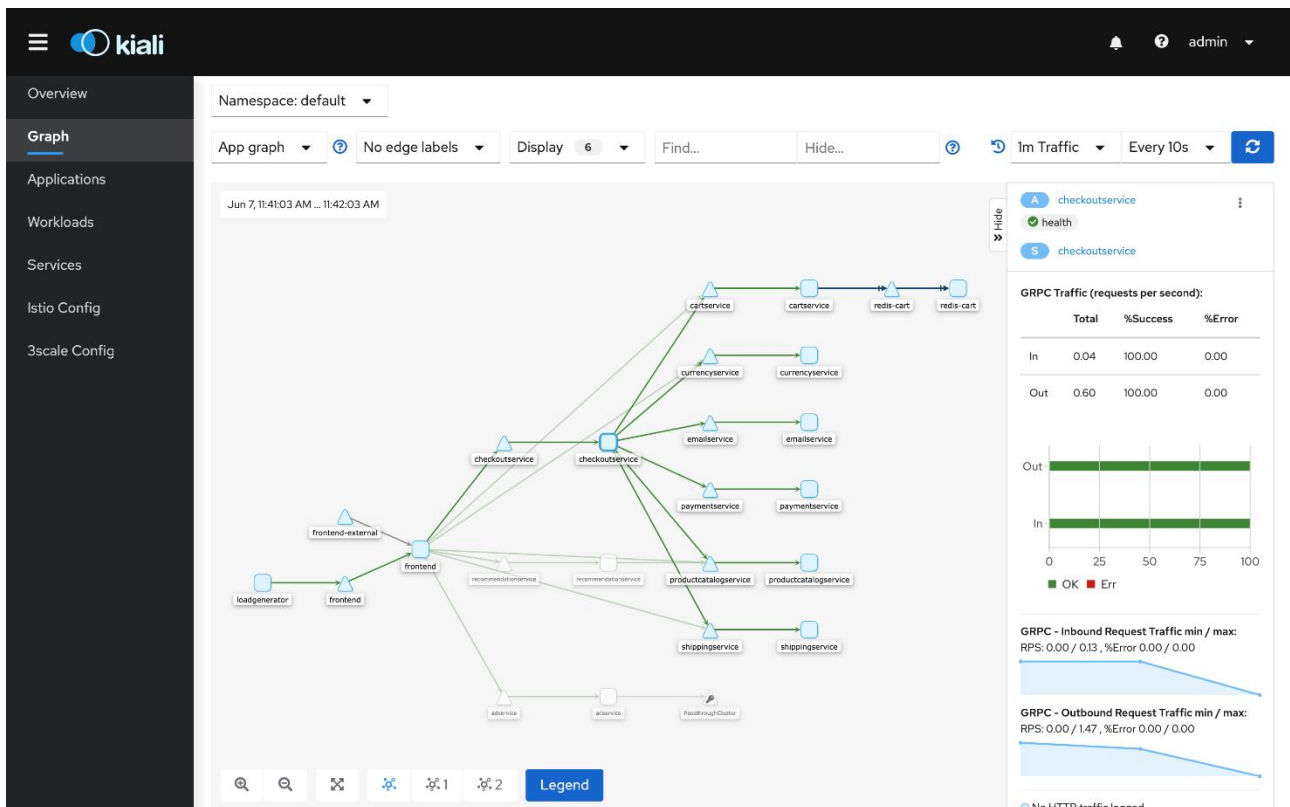
You will be presented with a service graph:



The graph is generated automatically by Kiali from service mesh metrics collected in Prometheus.

The nodes in the graph represent application services, whereas the edges reflect the communication paths between them. Gray edges indicate idle communication. Edges coloured in green indicate healthy communication. Edges coloured in a different color (yellow or red) indicate communication issues - we will evaluate such scenarios later in this workshop.

Now, click on the node representing the *checkout* service:

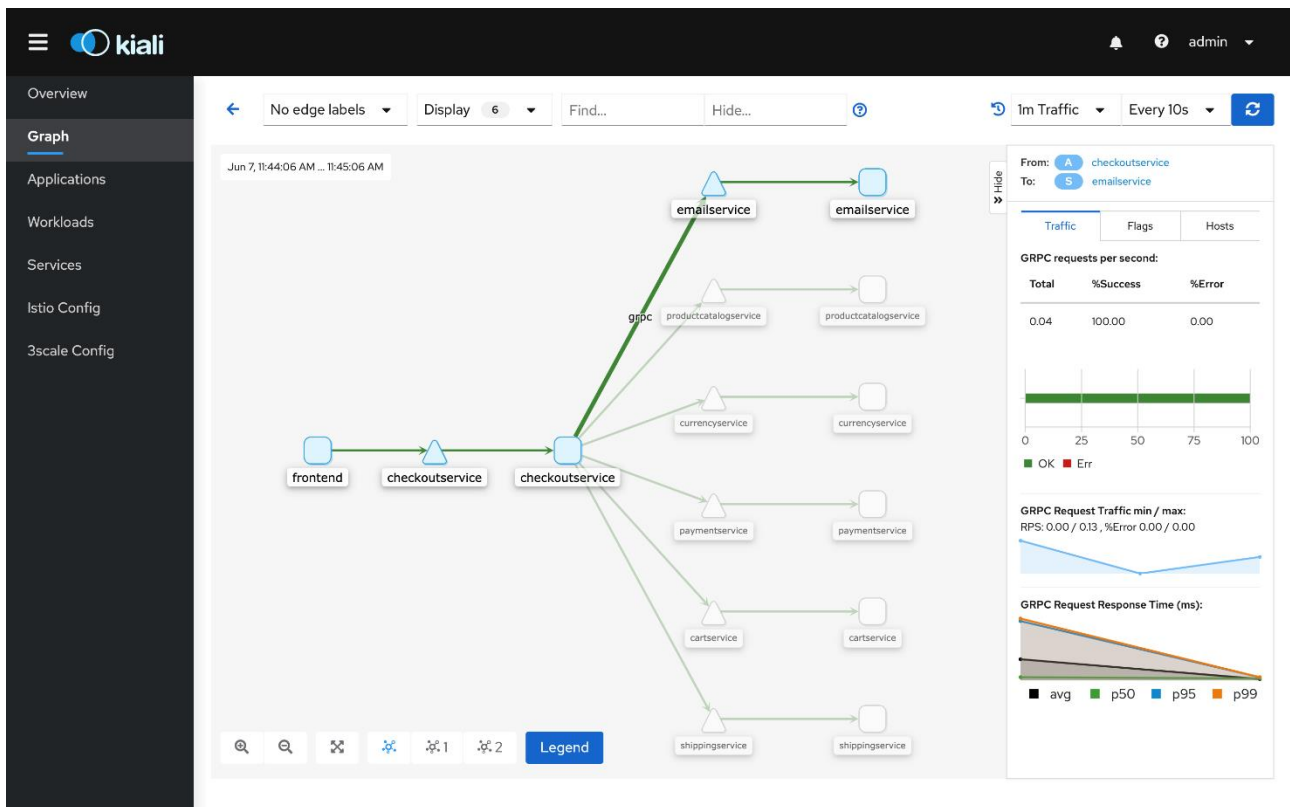


The graph should highlight the ingress and egress edges only for the checkout service. This way we can quickly lookup which services communicate with the selected service, and also which services the selected service communicates with.

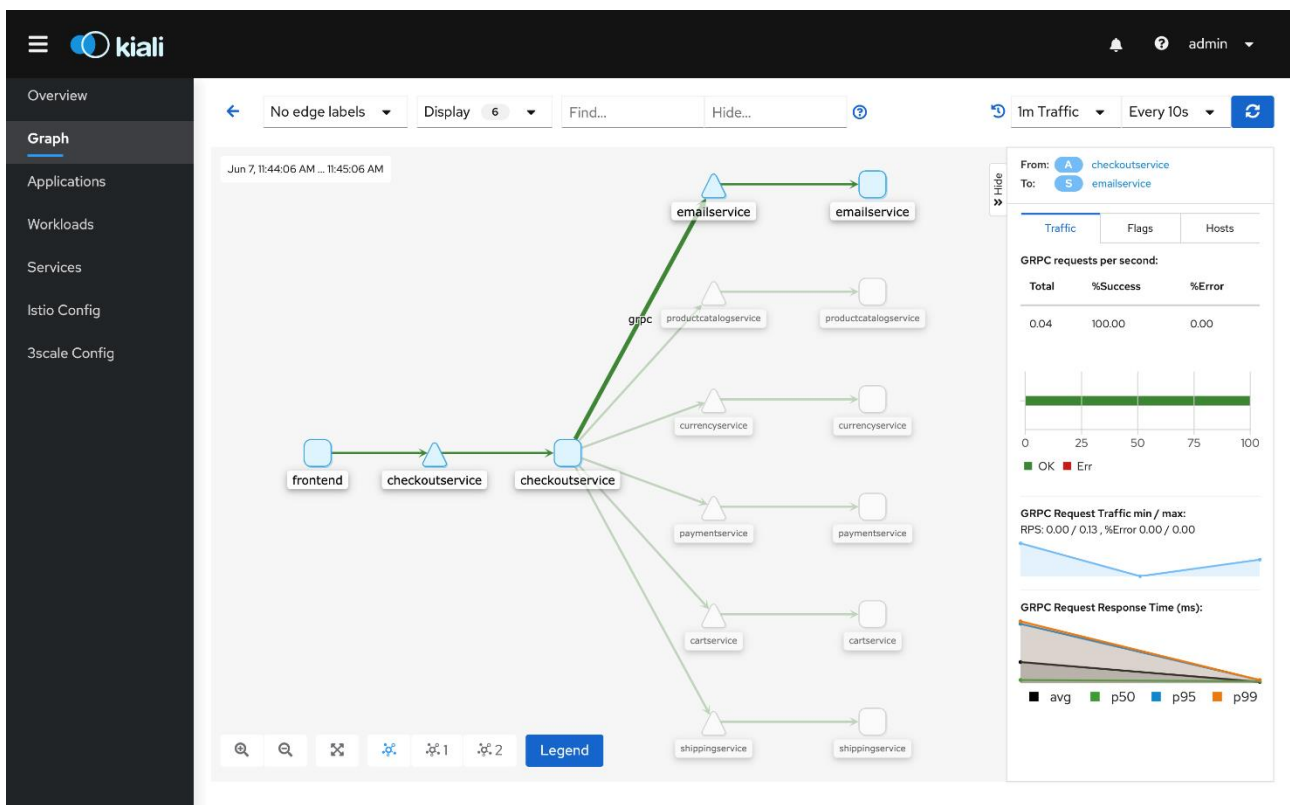
In addition, the dashboards presents basic communication metrics in the box on the right:

- operations per second (for ingress and egress traffic),
- success and error rates.

Double-click on the *checkout* node to retrieve the service subgraph with limited view of application services:



Now, click on the edge between the *checkout* and the *email* service:



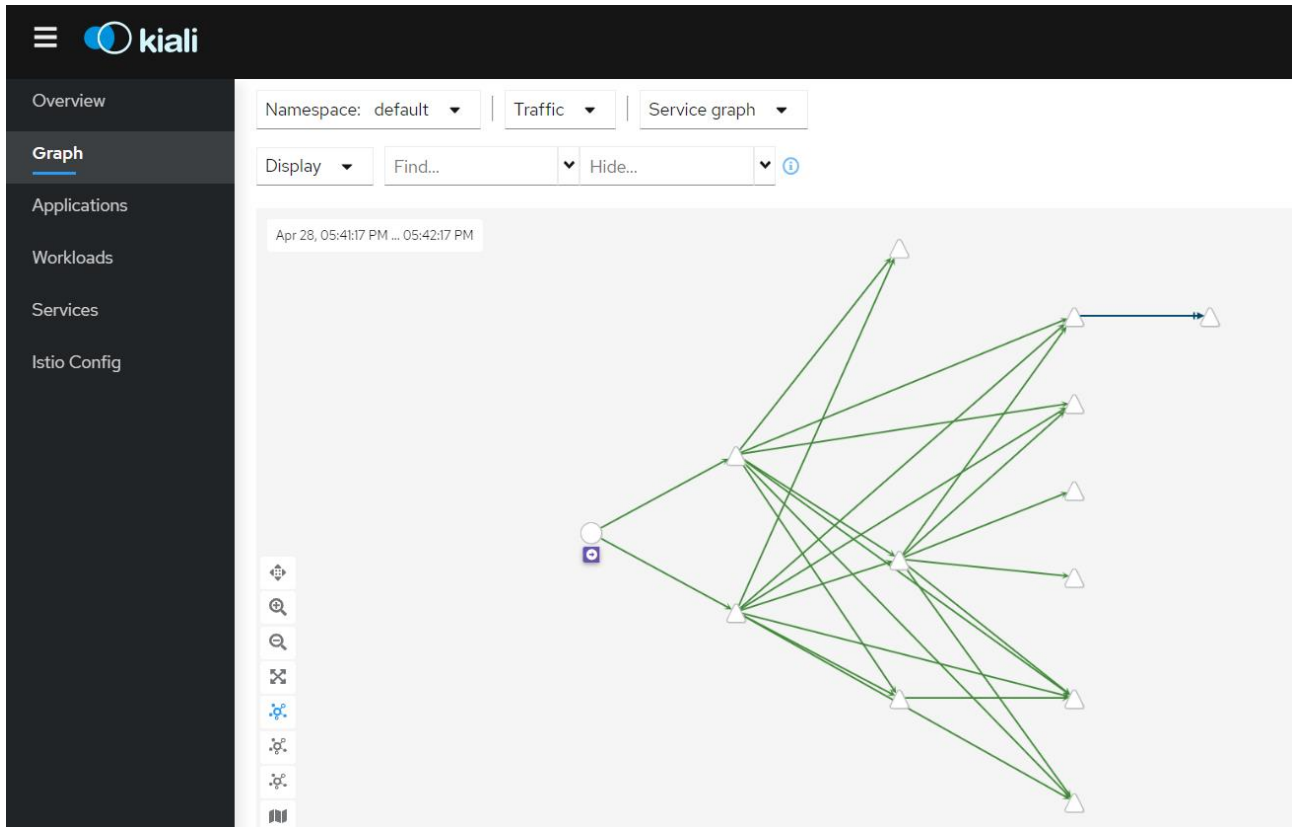
The edge should be highlighted and the dashboard should present the basic metrics for the communication path between these two services:

- used protocol (gRPC in this case),
- operations per second,
- success and error rates.

Go back to the full graph:

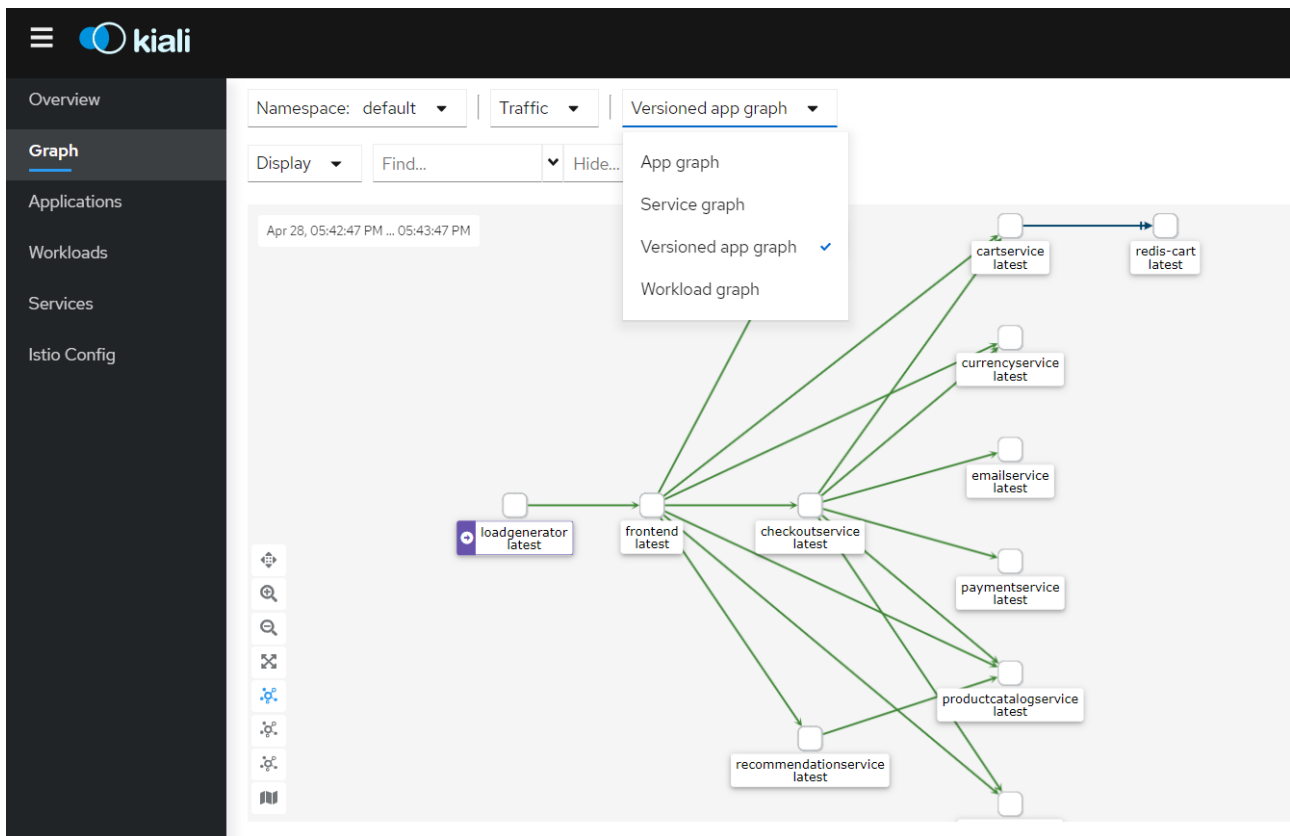


Switch to the *Service graph* view using the dropdown:



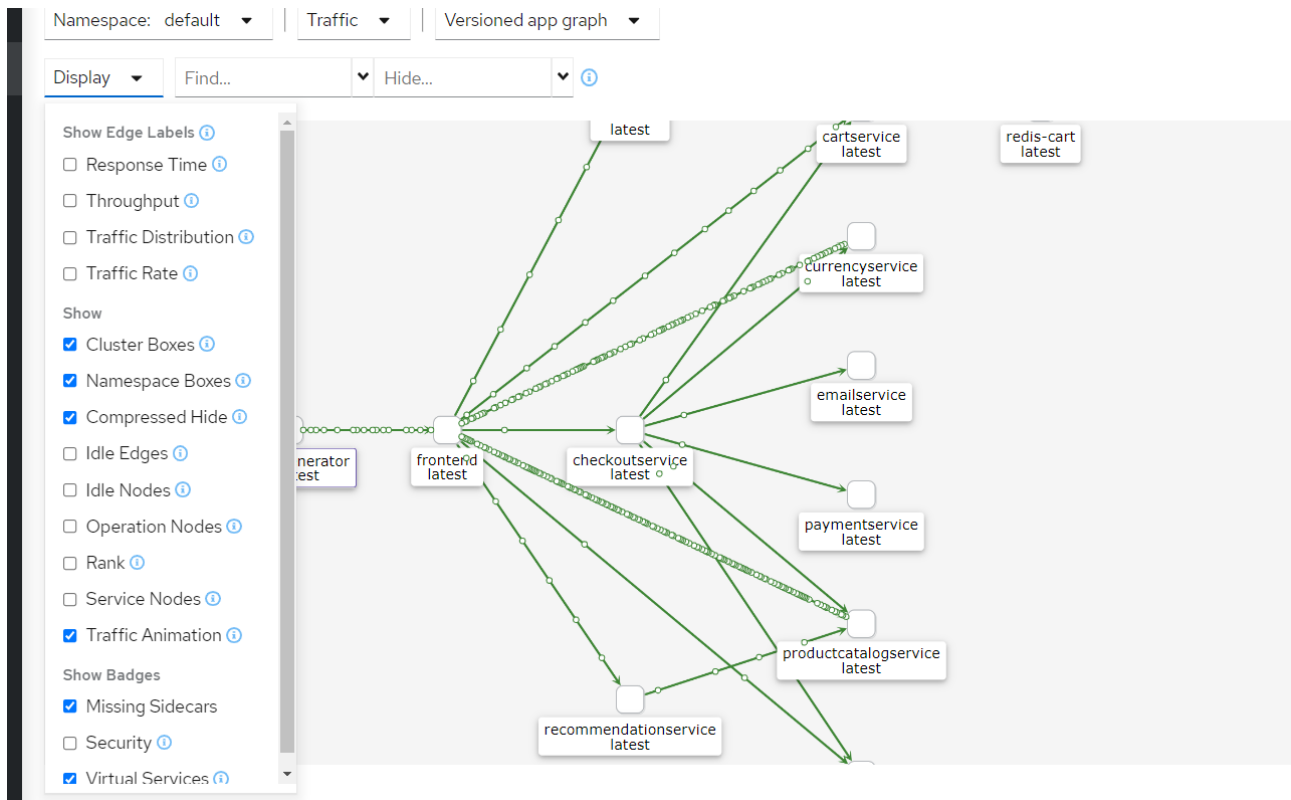
This view presents only services participating in the communication. There is no application context. The graph might serve as a simple lookup for identifying inter-service dependencies.

Switch to the *Versioned app graph* view:



This view presents inter-service communication along with traffic distribution between service versions. The graph is useful during the deployment of new service versions, e.g. canary releases.

Lastly, turn on *Traffic Animation* in the *Display* settings to get the real-time graph experience:



Small dots represent the traffic intensity. For intensive traffic, the dots are displayed denser and faster.

TASK 4 – Provide an answer to following questions and add it to your report

- Is the inter-service communication healthy? How do you check that?
- What is the entry-point service of the application?
- Which services communicate with the currency service?
- Which services communicate with the productcatalog service?
- Which services does the checkout service communicates with?
- What is the request volume (ops) for currency service? What are its success and error rates?
- What is the request volume (ops) for payment service? What are its success and error rates?
- What protocol is used on the communication path between the frontend and the recommendation service?
- What protocol is used on the communication path between the cart service and the *redis* service?

Task 5 – Traffic management, ingress gateway

Istio Gateway is a load balancer operating at the edge of the service mesh. It is in charge of controlling the ingress (inbound) and egress (outbound) traffic, allowing operators to specify which traffic should enter or leave the mesh. The gateway load balancer consists of one or many Istio

proxy instances that can be configured similarly to the proxies running alongside application services as sidecars. For configuring the gateway, Istio provides *Gateway* and *VirtualService* policy types.

Istio Ingress Gateway, as the name suggests, provides flexibility of Istio routing for the ingress traffic. Primarily, it enables setting the 4-6 load balancing properties such as ports to expose or TLS settings. It also supports application-layer traffic routing (L7), for instance, routing HTTP requests to application services based on the request path.

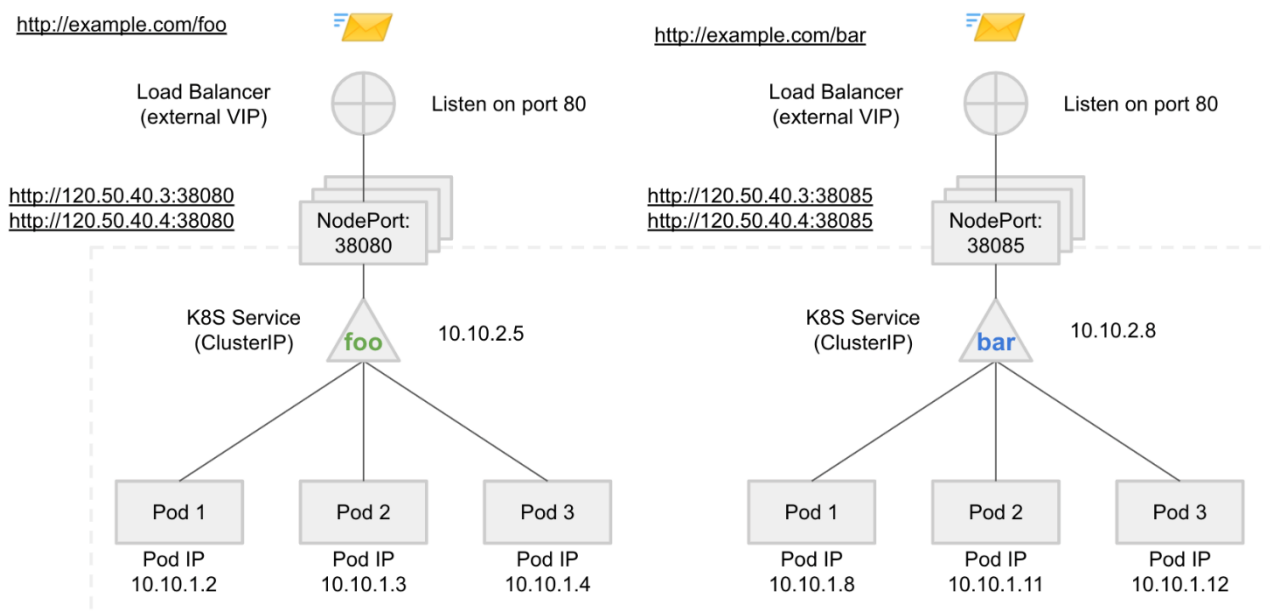
Istio Egress Gateway is another load balancer acting as a dedicated exit node for the traffic leaving the mesh. It enables limiting which services can access external networks, as well as enables securing the egress traffic with TLS policies. The Egress Gateway is out of the scope of this workshop.

In this workshop, we will configure Istio Ingress Gateway to expose the application to the public network.

Start with inspecting the Ingress Gateway service:

```
kubectl -n istio-system get svc istio-ingressgateway
```

Unlike other services of type *ClusterIP*, the Ingress service has the *LoadBalancer* service type. That means the service will be exposed outside the Kubernetes cluster. For such service type, AWS provisions an instance of an external ELB load balancer with a public IP address and a DNS hostname. The load balancer will be accessible by external clients (e.g., the web browser of a user) and will forward the ingress traffic from external networks to the Ingress Gateway service inside the Kubernetes cluster:



In order to obtain the address of the Ingress endpoint, run:

```
kubectl -n istio-system get service istio-ingressgateway -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}'
```

Above command displays the URL of Ingress Gateway – it will be used in communication with gateway.

By default, Istio proxy installed in the Ingress Gateway does not listen on any port - without proper configuration, client requests would fail with Connection Refused errors. First, apply the required gateway policy:

```
kubectl -n default apply -f templates/release/istio/frontend-gateway.yaml
```

Inspect its content:

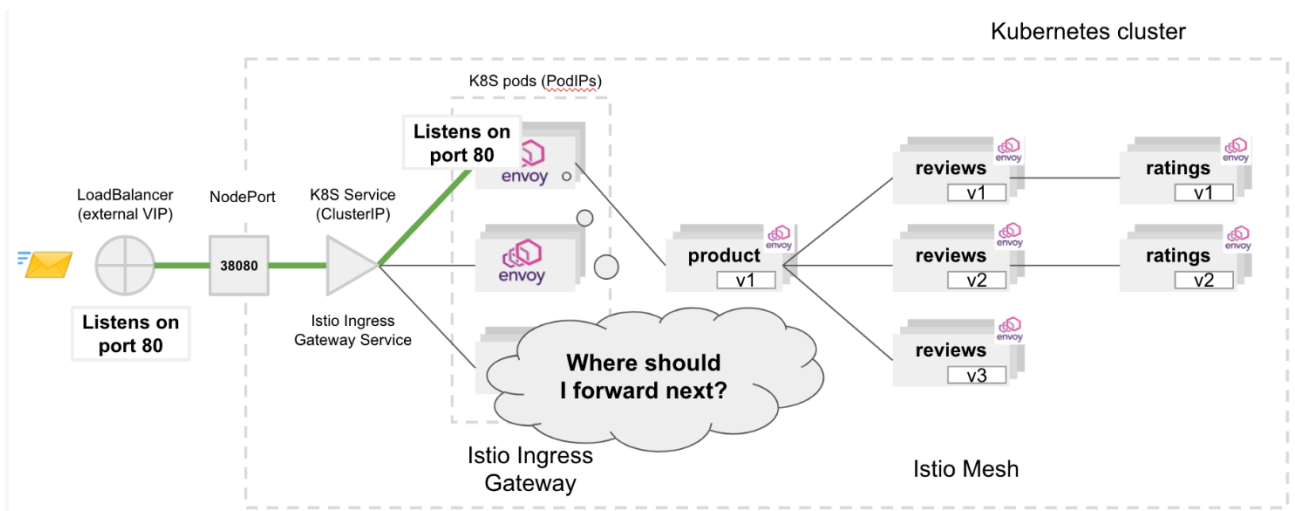
```
kubectl -n default describe gateway frontend-gateway
```

The above configuration opens port 80 for HTTP traffic for all destination services (Hosts: *) discovered in the mesh. The Selector binds the configuration to the specific Ingress instance.

Now, perform a HTTP request to the Ingress endpoint:

```
curl -v http://<INGRESS_GATEWAY_ADDRESS>
```

The gateway (*server: istio-envoy*) should respond with 404 Not Found status code.



Although Ingress has opened the access port for the inbound traffic, it does not know where to forward the request, hence the error. The destination service is not configured yet.

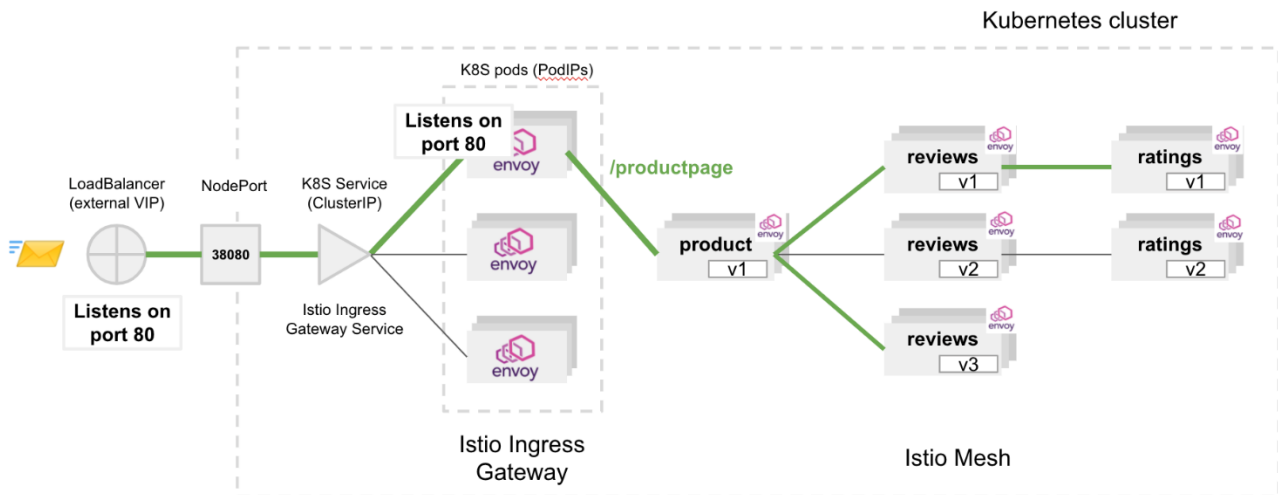
In order to instruct the gateway where to forward the request, apply the *VirtualService* policy:

```
kubectl -n default apply -f templates/release/istio/frontend-ingress.yaml
```

Inspect its content:

```
kubectl -n default describe vs frontend-ingress
```

First, the policy selects the gateway to configure (*frontend-gateway*). Then, it configures the gateway to route the entire HTTP traffic to the frontend service onto port 80.



Perform another HTTP request to the Ingress endpoint:

```
curl -v http://<INGRESS_GATEWAY_ADDRESS>
```

This time the request should succeed (HTTP/1.1 200 OK) and the Ingress should return the HTML content of the Online Boutique website.

Visit the Ingress address in your web browser (Chrome, Firefox) and explore the application. For instance, try to add products to the cart and go through the checkout process.

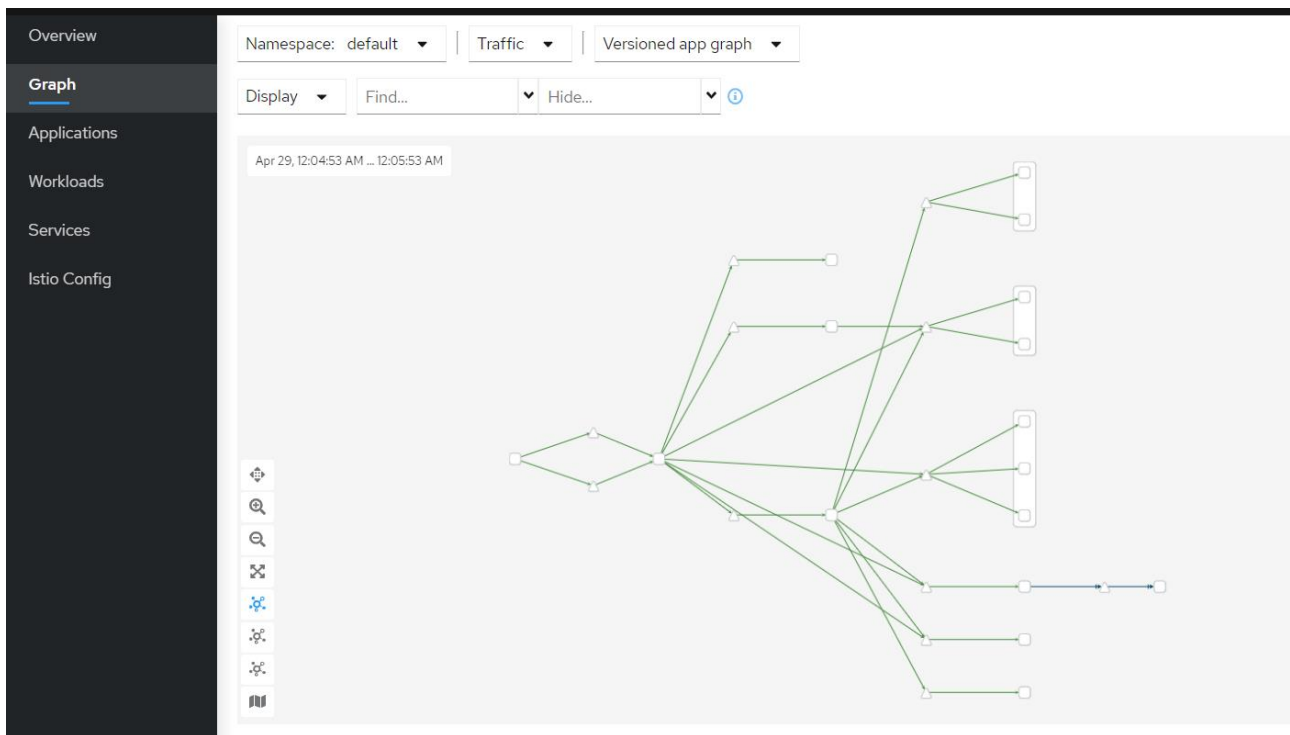
TASK 5 – add description of the virtual service policy and gateway policy to your report

Task 6 – Traffic management, traffic routing

First open the Kiali dashboard

```
istioctl dashboard kiali
```

Then, switch to the graph view and select *Versioned app graph* type from the graph dropdown. It should display a similar structure:



Note that due to multiple versions of some application services, traffic is distributed to all versions using Round Robin balancing algorithm. In case, each service version implements different business logic, balancing traffic to all versions can potentially lead to undesirable side effects and harm user experience. Typically, we should strive to handle user traffic only through one version of the service.

Now let's route traffic to a specific app version. Let's start with the *shippingservice*.

First, inspect the *shippingservice* deployments:

```
kubectl -n default describe deploy shippingservice-v1
kubectl -n default describe deploy shippingservice-v2
```

Note that pods produced by these deployments have labels determining the service version: *version=v1* and *version=v2*.

In order to enforce routing the user traffic to version v1 of the shipping service, apply the *DestinationRule* and *VirtualService* policies:

```
kubectl -n default apply -f templates/release/istio/shippingservice-dr.yaml
kubectl -n default apply -f templates/release/istio/shippingservice-vs.yaml
```

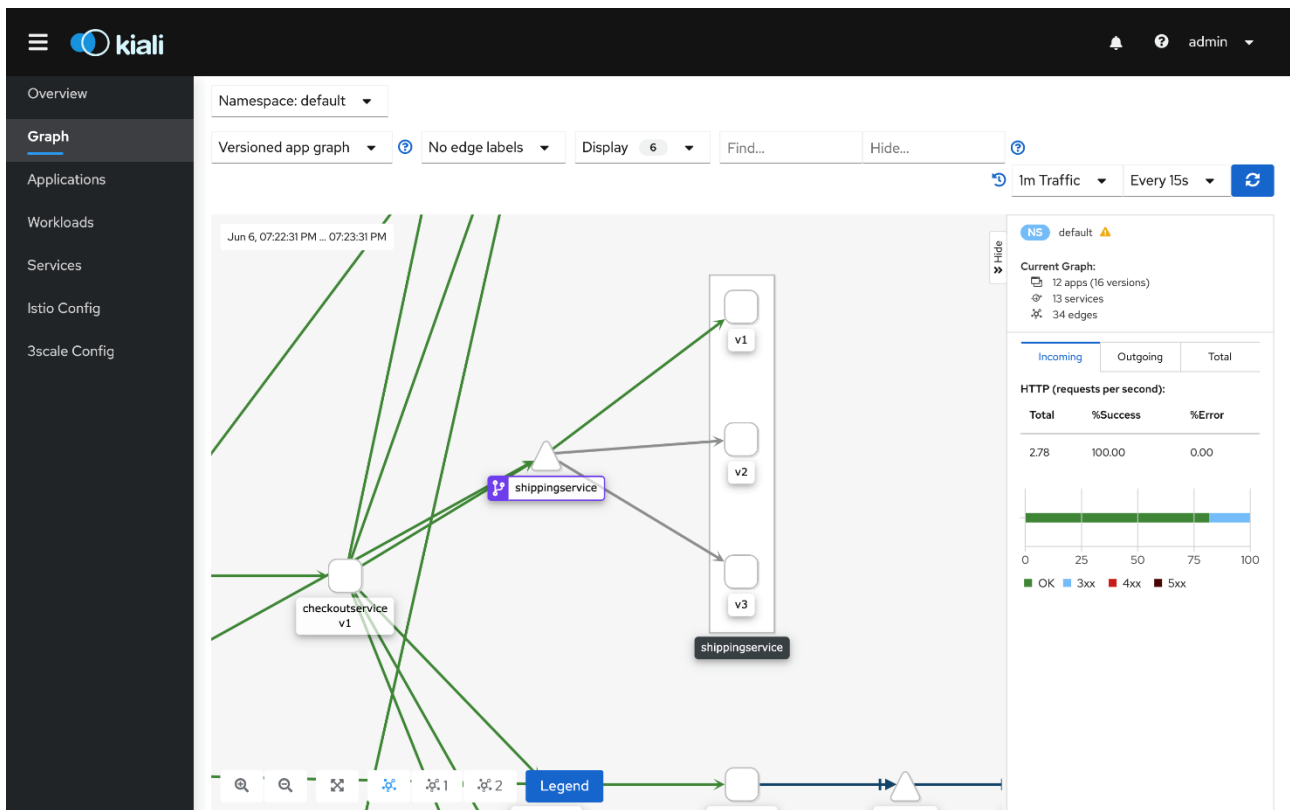
The former uses pod labels to specify named service subsets which group service endpoints by version:

```
kubectl -n default describe dr shippingservice
```

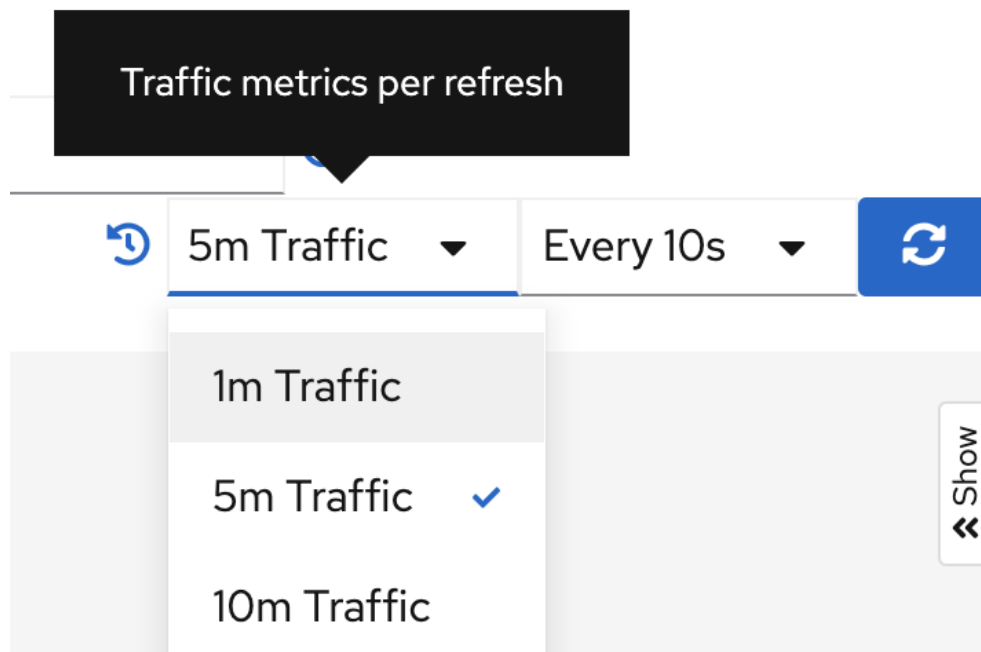
The latter uses the defined service subsets to route the traffic to the proper service version:

```
kubectl -n default describe vs shippingservice
```

After applying the policies, the traffic should be routed only to the version v1 of the *shipping* service:



It might take a while until the configuration is distributed to the proxies and Kiali infers new communication pattern from collected metrics. Changing the time range in the selector might reduce the time needed to obtain expected results in the graph:



The service node in the graph should be marked with a purple virtual service icon.

TASK 6 – add info regarding execution of following steps:

1. Apply *DestinationRule* and *VirtualService* policies to all services deployed in the service mesh regardless of how many versions each service provides. Route the traffic to version *v1* of each service.
2. Route *productcatalog* traffic to version *v2*.

Task 7 – Traffic management, traffic shifting

“Canary release is a technique to reduce the risk of introducing a new software version in production by slowly rolling out the change to a small subset of users before rolling it out to the entire infrastructure and making it available to everybody”¹

One of the benefits of using Istio is that it provides the control needed to deploy canary services. The idea behind canary deployment is to introduce a new version of a service by first testing it using a small percentage of user traffic, and then if all goes well, increase, possibly gradually in increments, the percentage while simultaneously phasing out the old version. If anything goes wrong along the way, we abort and rollback to the previous version. In its simplest form, the traffic sent to the canary version is a percentage of requests, but in more sophisticated schemes it can be based on the region, user, or other properties of the request.

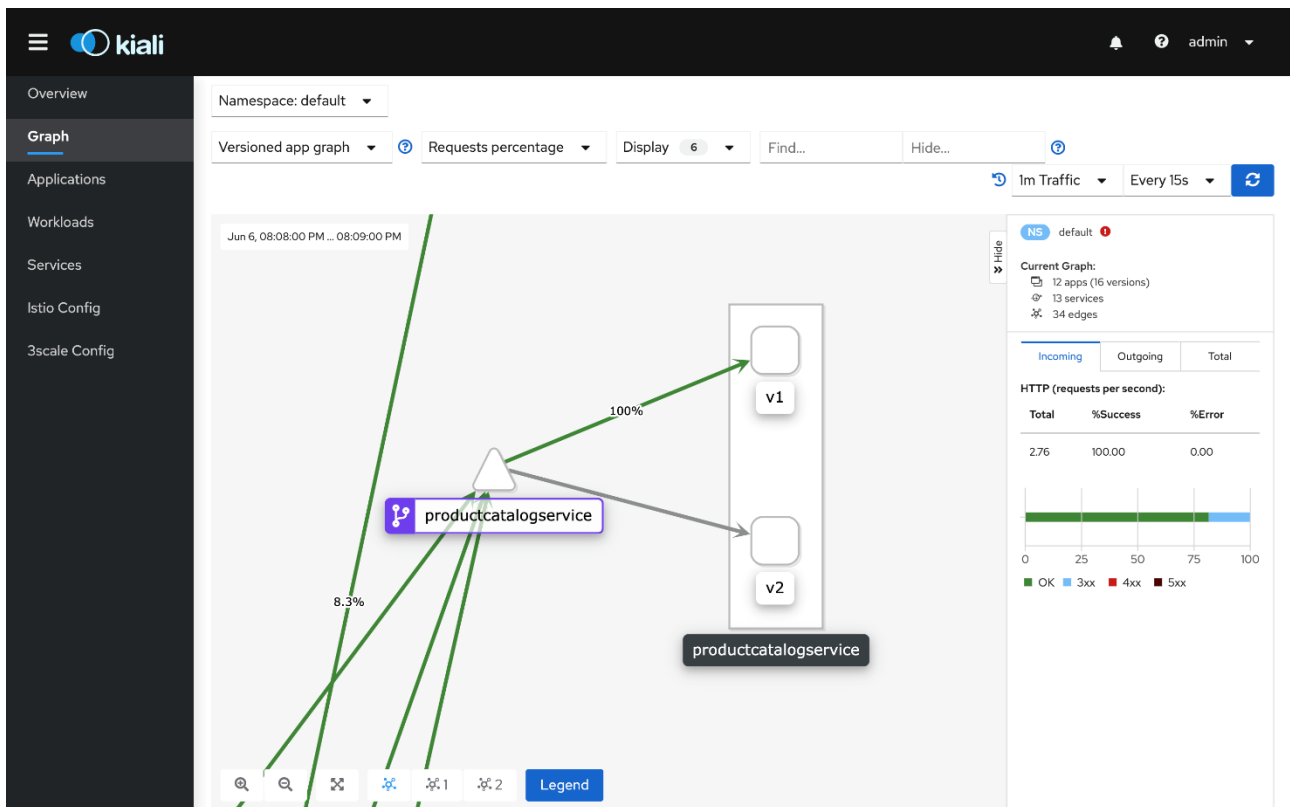
In this section we will conduct a canary release of the version *v2* of the *productcatalog* service.

Open the Kiali dashboard:

```
istioctl dashboard kiali
```

Then, switch to the graph view, select *Versioned app graph* type, and enable *Request percentage* in the edge label dropdown. The graph should display a similar structure:

¹ Martin Fowler



Note that initially all user traffic should be routed to the version v1 of the *productcatalog* service.

In order to start the canary deployment, apply the *VirtualService* policy:

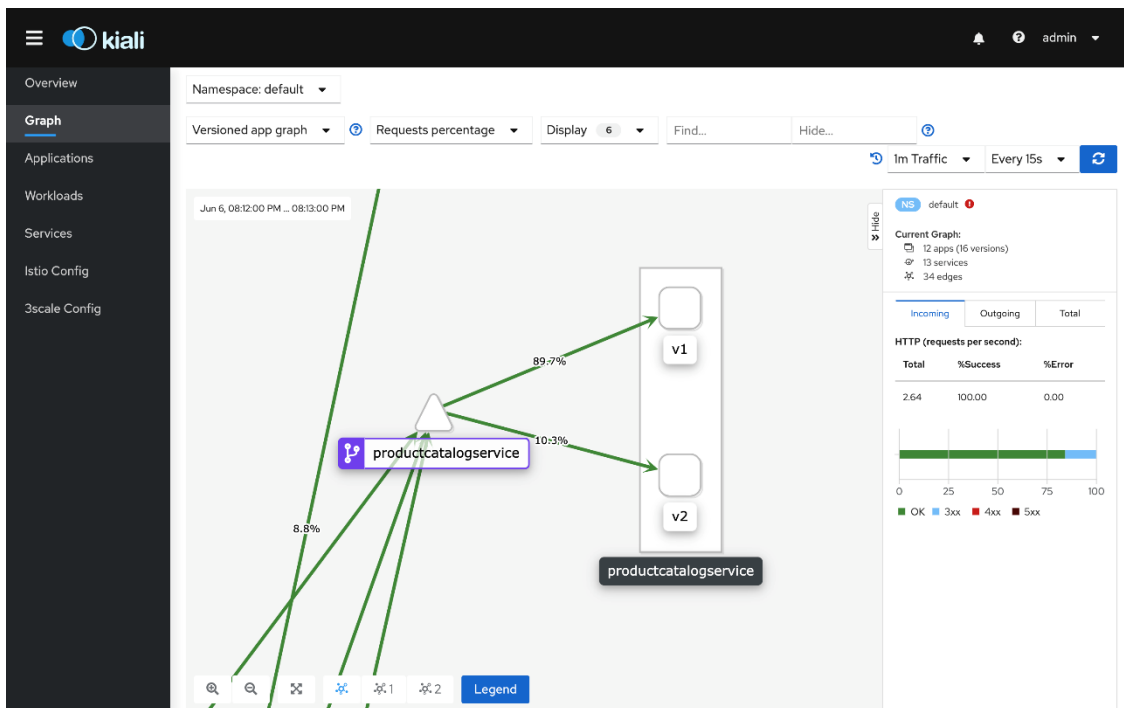
```
kubectl -n default apply -f templates/release/istio/productcatalogservice-vs-1.yaml
```

Inspect its content:

```
kubectl -n default describe vs productcatalog
```

The above policy enforces distributing the traffic between versions v1 and v2 of the productcatalog service in 9:1 ratio (Weight parameter). 90% of the traffic will be routed to the version v1. The remaining 10% will be routed to the version v2.

Observe the Kiali dashboard. After 1-2 minutes, the version routing should be reflected in the mesh:



In the meantime, we should continuously observe the essential health parameters for the new service workload (success rate, latency).

Open the Grafana dashboard:

```
istioctl dashboard grafana
```

Navigate to the **Mesh Dashboard** and observe the success rate for the v2 workload:

istio > Istio Mesh Dashboard		Last 5 minutes					5s
productcatalogservice.default.svc.cluster.local	productcatalogservice-v2.default	1.38 ops	275.42 µs	495.76 µs	89.17 ms	100.00%	
productcatalogservice.default.svc.cluster.local	productcatalogservice-v1.default	11.02 ops	286.89 µs	2.70 ms	78.29 ms	100.00%	

In case, something went wrong, we could immediately rollback the traffic to the old service version. However, the new service version performs well and we can route more user traffic towards it.

Apply the following policy:

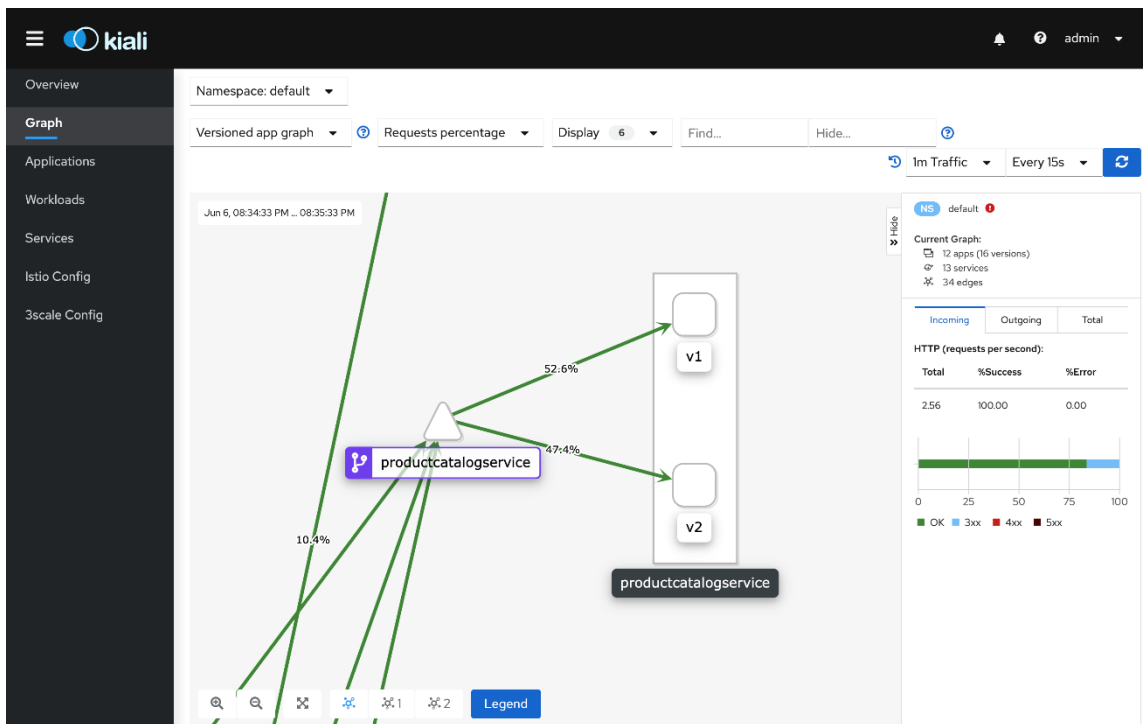
```
kubectl -n default apply -f templates/release/istio/productcatalogservice-vs-2.yaml
```

Inspect its content:

```
kubectl -n default describe vs productcatalog
```

This time we distribute the event portion of traffic to both service versions. 50% of the traffic will be routed to the version v1 and 50% of the traffic will be routed to the version v2.

In a while, the version routing should be reflected in the Kiali graph:



Again, we should inspect metrics in the Grafana dashboard to ensure that the communication to the new service version is healthy.

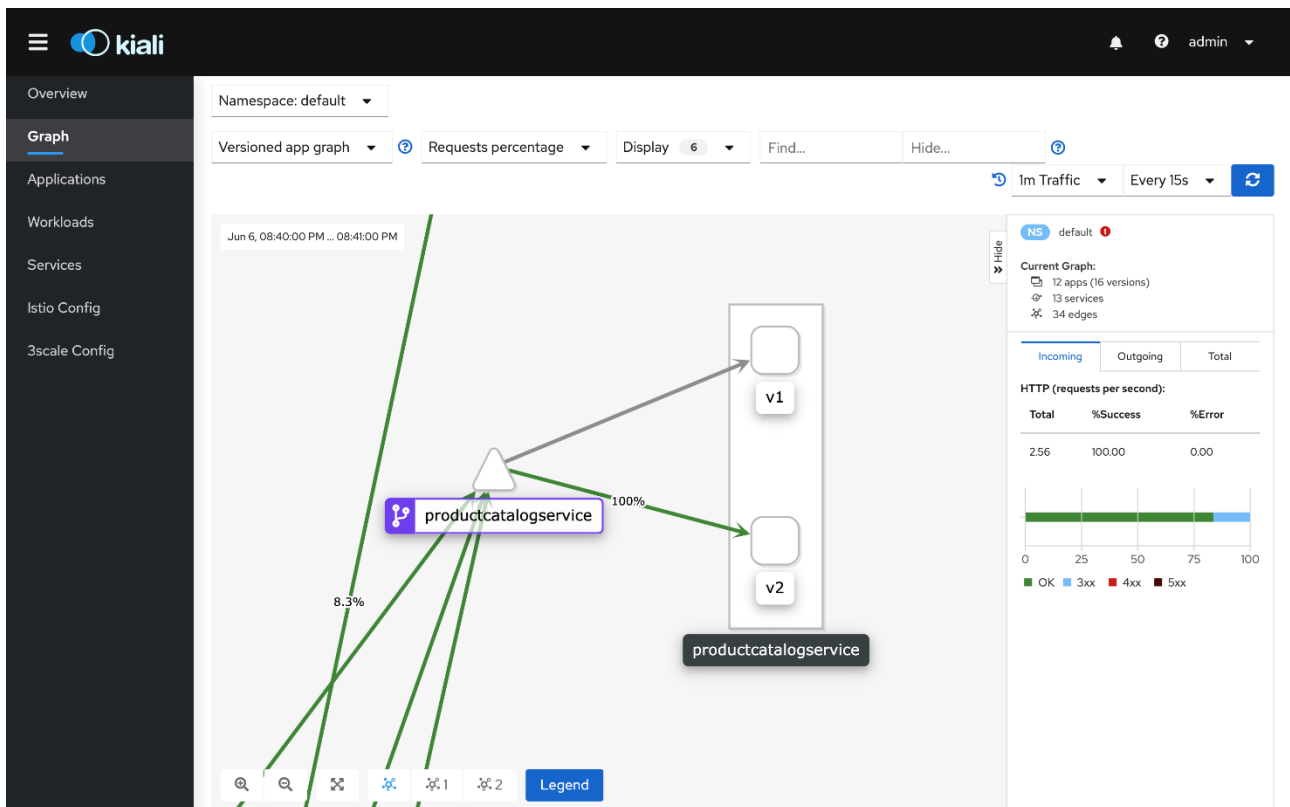
If the communication is healthy, apply the last policy to redirect 100% of the traffic to the version v2 of the productcatalog service:

```
kubectl -n default apply -f templates/release/istio/productcatalogservice-vs-3.yaml
```

Inspect its content:

```
kubectl -n default describe vs productcatalog
```

The canary deployment for the *productcatalog* service is complete:



TASK 7 – Conduct canary deployment for the v3 version of the shipping service. Include used Istio policies and screenshots from Kiali and Grafana in your report.

Task 8 – Traffic management, traffic shifting

Fault injection is a testing method that introduces errors into a system to ensure that it can withstand and recover from error conditions.

Unlike other mechanisms for introducing errors such as delaying packets or killing pods at the network layer, Istio proxy enables injecting faults at the application layer. That allows operators to inject more relevant failures, such as HTTP error codes.

Istio supports two categories of faults, both configured using a *VirtualService* policy:

- delays - timing failures, mimic increased network latency or an overloaded upstream service,
- aborts - crash failures, mimic failures in upstream services (HTTP error codes or TCP connection failures).

In this section, we will inject both types of faults into the test application and observe how it handles the failure.

In this scenario we will inject a delay fault into the payment service. Start with applying the fault policy:

```
kubectl -n default apply -f templates/release/istio/payment-service-fault-vs-1.yaml
```

Inspect its details:

```
kubectl -n default describe vs paymentservice
```

The above configuration injects 7s delay for 100% percent of the traffic to the payment service.

Now, visit the test application in your web browser. Order one item and proceed to the checkout stage

1 item in your cart

EMPTY CARTKEEP BROWSING

Google Cloud



Vintage Typewriter
SKU: #OLJCESPC7Z
Quantity: 1
USD 67.98

Shipping Cost: USD 4.72

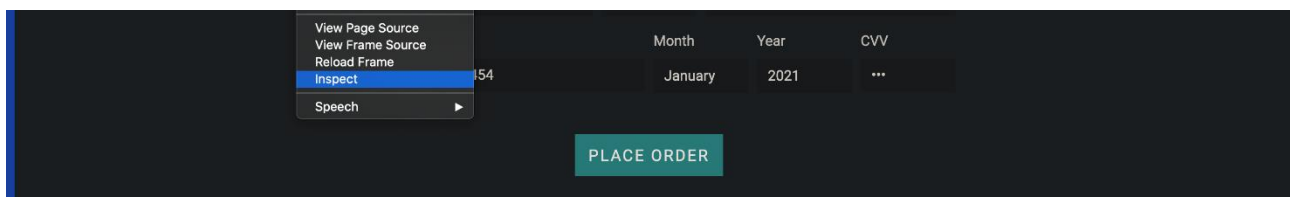
Total Cost: USD 72.71

Checkout

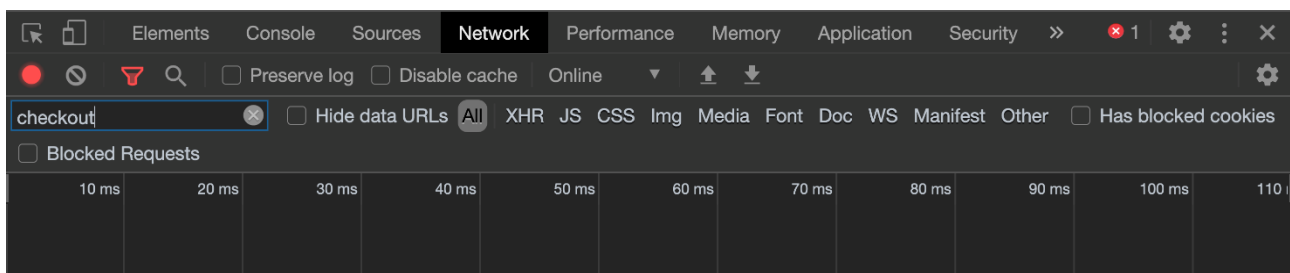
E-mail Address	Street Address	Zip Code	
someone@example.com	1600 Amphitheatre Parkway	94043	
City	State	Country	
Mountain View	CA	United States	
Credit Card Number	Month	Year	CVV
4432-8015-6152-0454	January	2021	...

PLACE ORDER

Before placing the order, open the network analyzer tool provided by your web browser:



Filter out the requests to the checkout service:



Place the order. The operation should complete with the 7s delay:

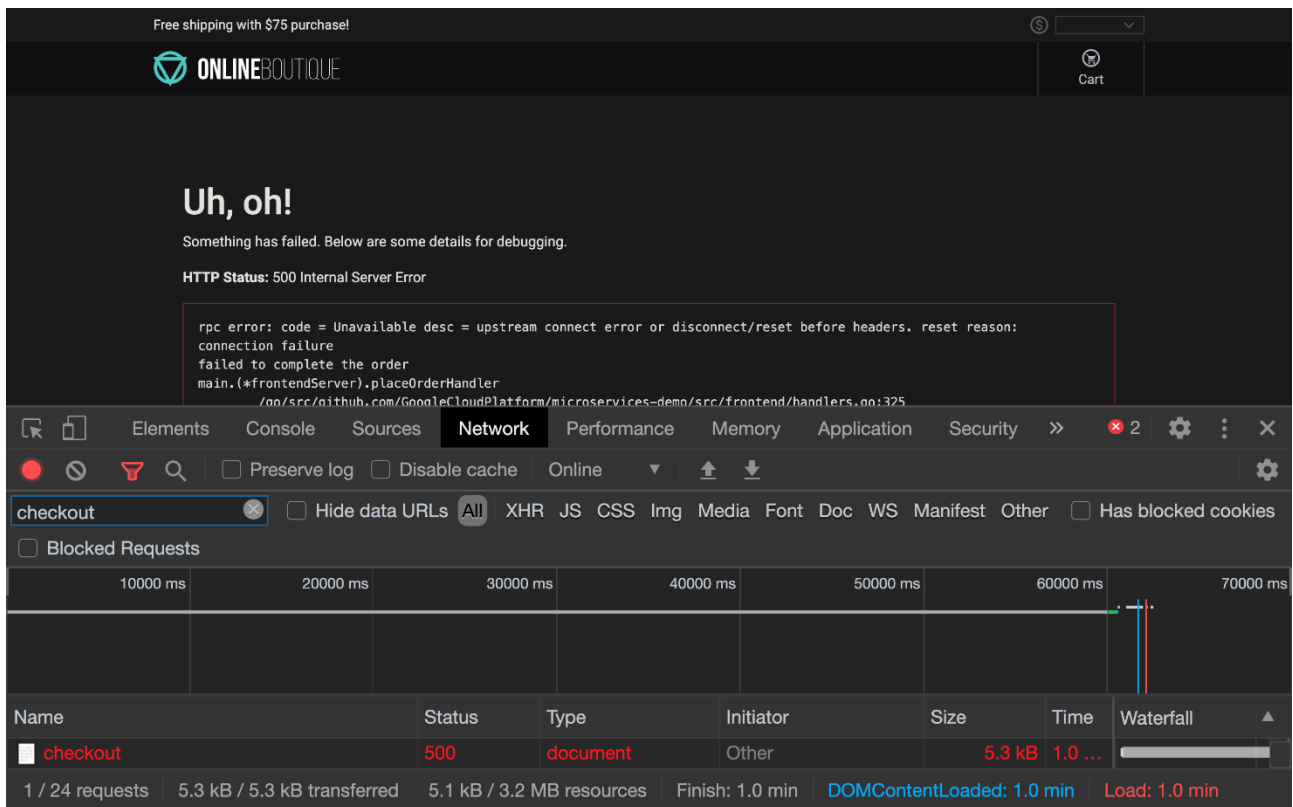
The screenshot shows a web browser with the URL `https://www.onlineboutique.com`. The page displays "Your order is complete!" with the Online Boutique logo. The Chrome DevTools Network tab is open, showing a single request named "checkout". The request details show a status of 200, type of document, and a time of 7.74 s. The waterfall view shows a green bar representing the request duration. The bottom status bar indicates "1 / 199 requests", "7.9 kB / 7.9 kB transferred", "7.6 kB / 3.9 MB resources", "Finish: 10.21 s", "DOMContentLoaded: 9.13 s", and "Load: 9.66 s".

The checkout service communicates with the payment service, hence the delay occurred.

The 7s delay was handled gracefully by the application. Let's rise the delay to 120s to emulate a severe network congestion:

```
kubectl -n default apply -f templates/release/istio/payment-service-fault-vs-2.yaml
```

Add another item and place the order. This time the application should fail and expose the 500 Internal Server Error to the user:



In production environment, applications must always hide internal errors from the end user. In addition, the application should configure a request timeout.

Next we will inject an abort fault into the productcatalog service.

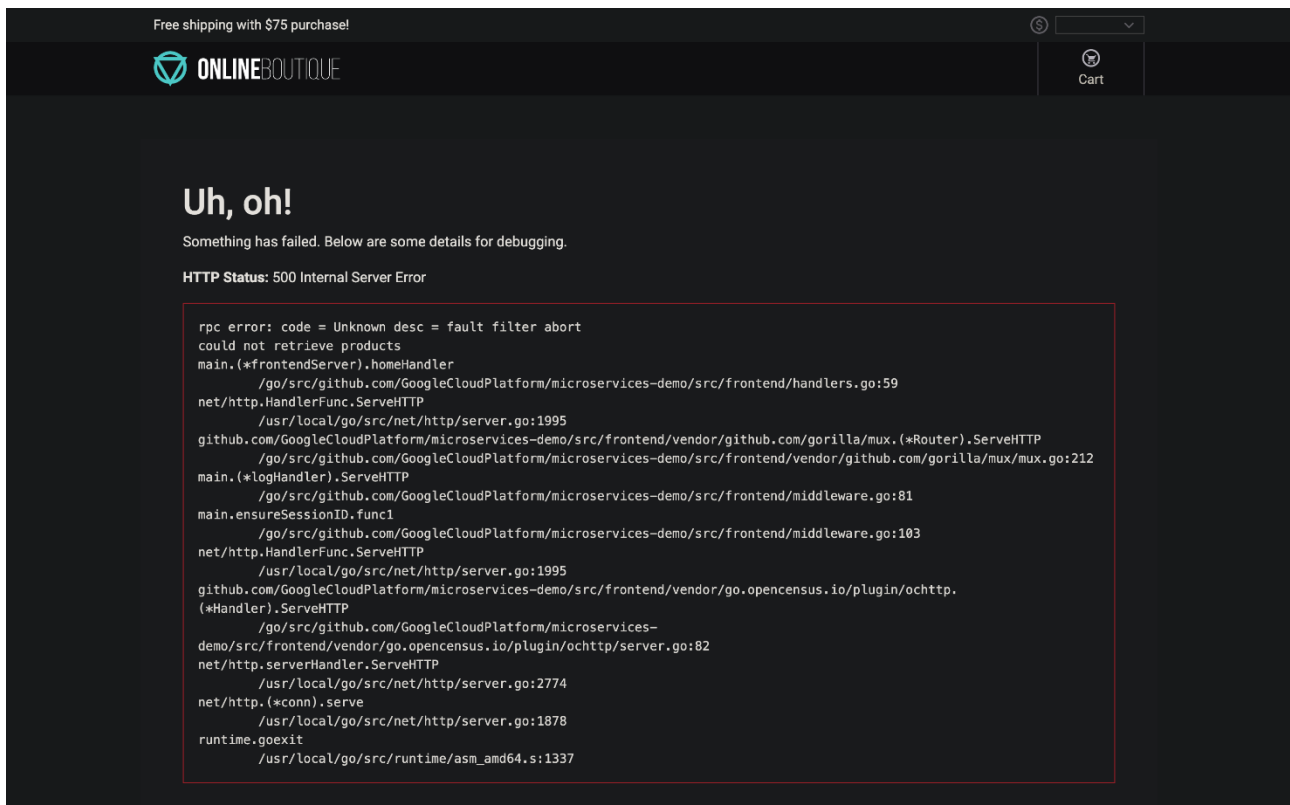
Apply the following fault policy:

```
kubectl -n default apply -f templates/release/istio/productcatalogservice-fault-vs-1.yaml
```

Inspect its details:

```
kubectl -n default describe vs productcatalogservice
```

The above configuration injects 500 error for 30% percent of the traffic to the productcatalog service. Refresh the main page of the test application several times to observe the error message exposed to the user:



Again, the application should hide the internal error from the user and instead display a gentle warning, for instance:

Product Catalog is temporarily unavailable. Sorry for inconvenience.`

TASK 8 – Add screen with the Error message from the website

Task 9 – Stopping the cluster

At the end of exercise stop the lab by resetting account in the Vocareum service (end lab is not enough !):